

UR-CP-ABE: CP-ABE With Flexible Construction Mechanism and Efficient User Revocation Capability for Access Control in the Cloud

Zhen Guo , Jiangkai Gao, Shuainan Liu , Rong Wang , Chaosheng Feng , Keping Yu , Senior Member, IEEE, Kim-Kwang Raymond Choo , Senior Member, IEEE, and Mohsen Guizani , Fellow, IEEE

Abstract—Ciphertext-Policy Attribute-Based Encryption (CP-ABE) schemes with user revocation allow for dynamic updates to users' access rights. However, existing schemes often encounter issues such as low re-encryption efficiency, inflexible access control, and susceptibility to collusion attacks. To address these challenges, we propose UR-CP-ABE, an efficient user revocation scheme built upon a double encryption method. Specifically, UR-CP-ABE stores both valid and revoked users' identity information in a binary tree. This design enables flexible user authority revocation by modifying only the binary tree's relevant secret sub-items. Moreover, the scheme limits the scope of revocation-induced binary tree updates to a single sub-item. This key optimization resolves the critical issue where re-encryption overhead scales linearly with the number of attributes of revoked users. In addition, we eliminate the possibility of attackers constructing secret sub-keys, preventing the collusion attacks. UR-CP-ABE also supports bidirectional revocation, allowing for the revocation and restoration of user rights, which is not available in other related schemes. Our theoretical analysis and experiments demonstrate that UR-CP-ABE has better performance than other schemes, especially in the re-encryption stages. Moreover, UR-CP-ABE has been proved to be secure based on the decisional q -parallel BDHE hardness assumption in the standard model.

Index Terms—CP-ABE, access control, user-level, revocation, bidirectionality.

Received 5 March 2025; revised 8 December 2025; accepted 29 January 2026. Date of publication 3 February 2026; date of current version 12 May 2026. This work was supported in part by the Natural Science Foundation of Sichuan Province under Grant 2022NSFSC055 and Grant 2023NSFSC1397 and in part by JSPS KAKENHI under Grant JP25K00373. The work of K.-K. R. Choo was supported by Cloud Technology Endowed Professorship. (Corresponding author: Chaosheng Feng.)

Zhen Guo is with the School of Computer Science, Sichuan Normal University, Chengdu 610101, China, and also with the College of Computer Science, Sichuan University, Sichuan 610065, China (e-mail: gz@scu.edu.cn).

Jiangkai Gao, Rong Wang, and Chaosheng Feng are with the School of Computer Science, Sichuan Normal University, Chengdu 610101, China (e-mail: 18881286786@163.com; rwang@sicnu.edu.cn; csfenggy@sicnu.edu.cn).

Shuainan Liu is with the Department of Computer Science, Texas Tech University, Lubbock, TX 79409 USA (e-mail: Shuainan.Liu@ttu.edu).

Keping Yu is with the Graduate School of Science and Engineering, Hosei University, Tokyo 184-8584, Japan (e-mail: keping.yu@ieee.org).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, University of Texas at San Antonio, San Antonio, TX 78249 USA (e-mail: raymond.choo@fulbrightmail.org).

Mohsen Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Masdar City SE45 05, UAE.

Digital Object Identifier 10.1109/TDSC.2026.3660271

I. INTRODUCTION

CLOUD computing has revolutionized data management by providing scalable storage solutions and powerful big-data processing capabilities. This shift to cloud-based storage, however, introduces new challenges—particularly in securing outsourced data and enforcing efficient access control. In 2005, Sahai and Waters introduced the Attribute-Based Encryption (ABE) scheme, building on the concept of fuzzy identity encryption [1]. The ABE framework involves key entities such as the authority and data owner, and comprises four fundamental algorithms: *Setup*, *KeyGen*, *Encrypt*, and *Decrypt*. This approach encrypts data in line with predefined access policies prior to cloud upload, converting it to ciphertext for secure storage. User secret keys are then issued according to users' attributes, forming a comprehensive encryption-decryption system that ensures robust access control and facilitates secure data sharing and publication. ABE is commonly categorized into two schemes: Key-Policy Attribute-Based Encryption (KP-ABE) [2] and Ciphertext-Policy Attribute-Based Encryption (CP-ABE) [3]. In CP-ABE, the access policy is embedded in the ciphertext, making it particularly suitable for modern public cloud environments by providing enhanced flexibility for access control and secure data sharing.

As researchers on CP-ABE has deepened, several challenges have come to light. In particular, once the attribute set for all users has been computed, any modification to a user's attributes requires a complete recalculation of the system [4]. Changes in user attributes correspond to alterations in access permissions—a process known as user revocation. For example, as shown in Fig. 1, in a medical system, it is crucial to promptly revoke access rights of former employees to protect sensitive patient data. Similarly, when an associate chief physician is promoted to chief physician, their access rights must be updated accordingly. Ensuring real-time access control in CP-ABE therefore poses significant challenges. Developing an efficient revocation mechanism within a certificate-free CP-ABE framework is a complex and computationally demanding task [5].

To address changes in user-level access rights, two main approaches have been explored: direct revocation and indirect revocation. Direct revocation embeds a list of revoked users directly into the encrypted file, thereby revoking access for all users on that list. By contrast, indirect revocation involves the

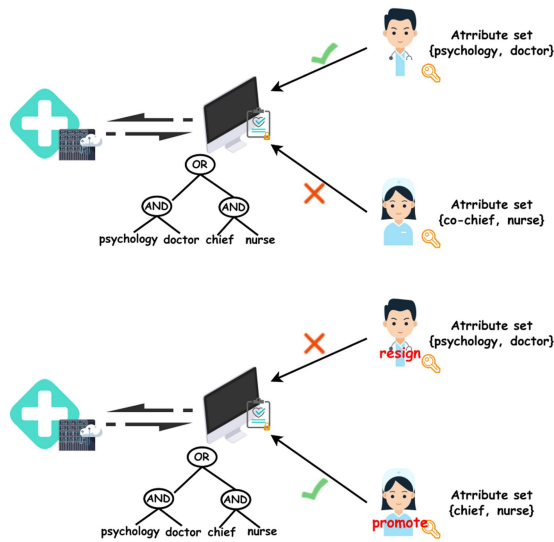


Fig. 1. Application of CP-ABE in medical systems.

authorization authority either real-time or periodically generating and distributing new keys for non-revoked users, with the encrypted file being re-encrypted accordingly. Based on these two approaches, many solutions [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26] have been proposed in an attempt to implement an effective revocation mechanism within CP-ABE framework. However, existing solutions suffer from three critical limitations:

- *Re-encryption efficiency bottleneck:* The computational cost scales linearly with the number of attributes associated with revoked users. This limits real-time secure data sharing, and re-encryption overhead escalates as the system expands.
- *Inflexible revocation mechanisms:* Adjusting user access needs repeated re-encryption or key regeneration. Even a single user's access change demands extra computations on non-revoked users' keys, making the process resource-intensive.
- *Security risks in optimization:* While flexible access control often demands algorithm tweaks, modifying keys or ciphertext may introduce vulnerabilities. Altered or new key components could expose critical info and raise collusion attack risks.

Thus, a low-cost, large-scale user-revocation solution with low computation and storage has great practical value.

A. Motivations

Existing CP-ABE mechanisms supporting direct revocation can protect data confidentiality and enable outsourced encrypted data sharing. Nonetheless, they still have much room for improvement. Our goal is to design a CP-ABE scheme that:

- develops a flexible encryption & decryption mechanism that prevents re-encryption overhead from increasing linearly with the number of revoked users.
- offers protection against collusion attacks, enhancing the flexibility of access structures and eliminating the possibility for attackers to construct secret sub-keys.

- supports bidirectional revocation, allowing for efficient revocation and restoration of user permissions.

B. Related Research

To achieve changes in user-level access rights, many studies have explored diverse strategies. We compare representative approaches with ours in Table I.

Boldyreva et al. [4] proposed a new model to address IBE revocation limitations. They defined revocable IBE primitives and security models, and combined the fuzzy IBE structure [1] to reduce the user update range from linear to logarithmic. The scheme assigns a time attribute to cloud ciphertexts; upon revocation, revoked users are listed, and the authority periodically updates non-revoked users' private keys to block revoked users from decrypting. Attrapadung et al. [6] aimed to solve indirect revocation bottlenecks by extending identity-based multicast encryption to support user-level revocation; however, this coarse-grained method requires revoking all user attributes to change access rights, lacking flexibility and efficiency. Xu et al. [7] designed a dynamic user revocation solution where the cloud server recalculates sub-keys via delegated keys, but it fails to support attribute revocation—if some attributes are revoked, a user cannot decrypt any confidential files even with other valid attributes.

Balu et al. [8] proposed a direct user access revocation scheme whose its encryption phase takes both access policies and a revocation list (the key to enabling revocation and restricting decryption). Tysowski et al. [9] designed a CP-ABE user revocation scheme with heavy encryption, grouping users at initialization and assigning group keys; however, collusion between an existing user and a revoked user can bypass protection in some cases, endangering confidentiality. Shi et al. [10] proposed a scheme where a third party undertakes re-encryption computations, with an added verification link to ensure third-party honesty. However, it does not support two-way revocation, so revoked users cannot restore decryption capabilities. Liu et al. [11] innovatively designed a scheme supporting direct user access revocation and traceability, which avoids user-collusion but lacks forward security. Ma et al. [12] developed a large-attribute scheme with direct revocation and verifiability: it implements revocation by updating the revocation list, uses subset coverage to improve broadcast efficiency, assigns ciphertext updates to the cloud server, and adds a correctness verification process.

Liu et al. [13] released the TR-CPABE scheme for fast user revocation, which combines user binary tree and CP-ABE features to achieve flexible permission revocation and user-level access recovery but has fatal security loopholes—exposed secret sub-keys in private keys leave it vulnerable to user-collusion. Yasumura et al. [14] focused on integrating proxy re-encryption with attribute revocation, with many researchers opting for proxy servers to handle revocation tasks. Ramu et al. [15] proposed an enhanced CP-ABE scheme with user revocation for cloud outsourced data systems, achieving fine-grained revocation via immediate attribute modification. Zhao et al. [16] designed a revocable publish-subscribe scheme supporting multiple publishers/subscribers while protecting data from unauthorized users and cloud servers. Lian et al. [17] conceived a large-attribute

TABLE I
COMPARISON AMONG EXISTING CP-ABE SCHEMES SUPPORTING REVOCATION MECHANISMS

Schemes	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[17]	[18]	[19]	[20]	[21]	[22]	[23]	[24]	[15]	[16]	[25]	[26]	Ours
Rev. type	①	①	①	①	①	①	②	①	②	②	②	①	②	①	①	②	①	②	①	①
Key update	×	×	×	×	×	×	×	×	×	✓	✓	×	✓	×	×	✓	×	✓	×	×
Re-encryption	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Re-en. outsourcing	×	×	✓	×	✓	×	×	×	×	×	×	×	✓	×	✓	×	×	×	✓	✓

¹ Revocation type: ①direct, ②indirect.

CP-ABE with user revocation that splits master-key sub-items between cloud servers and data consumers for two-part re-encryption, but the efficiency is low.

To further address efficiency, security, and privacy in ABE revocation, researchers have proposed new ideas. Sethi et al. [18] published a cryptosystem combining attribute encryption, decentralization and multi-authority, integrating large attribute domains with expressive policies to solve user-level revocation and forward-backward secrecy, with practical effective revocation. Hao et al. [19] proposed a cloud-assisted IoMT data sharing scheme combining proxy re-encryption and key obscuration, enabling cloud servers to efficiently handle ciphertext re-encryption and user key updates. Deng et al. [20] designed a practical CP-ABE-based access control scheme, allowing Trusted Authority to manage user credentials efficiently and incorporating user binary trees and verifiable ciphertext delegation for attribute and user revocation, but it faces single-point failure risks in various scenarios. Chen et al. [21] proposed a revocable attribute-based encryption scheme to enhance data integrity protection.

Lu et al. [22] proposed a fine-grained access control scheme for attribute private key distribution, using Proxy Re-Encryption to facilitate access permission updates. Liang et al. [23] developed an attribute-based encryption framework for smart healthcare privacy-preserving authentication, with flexible access revocation to block malicious users from decrypting Electronic Health Records. Li et al. [24] studied Hardware-based Revocable Attribute-Based Encryption to eliminate ciphertext delegation and mitigate collusion between untrusted storage servers and revoked users. Sarma et al. [25] designed an encryption-based CP-ABE method with fine-grained access control and user revocation, where resource-constrained users only need to store a constant lightweight decryption key. Ge et al. [26] proposed an Attribute-Based Proxy Re-Encryption with Direct Revocation for encrypted data sharing, enabling cloud servers to directly revoke users from the re-encryption key-linked original sharing set.

In summary, it is of significant practical value to realize a flexible and efficient CP-ABE scheme that supports user revocation while maintaining low computational and storage costs.

C. Collusion Attack

A typical security vulnerability in current revocation schemes is collusion attacks. Take [13]'s TR-CPABE scheme as an example: it combines a user binary tree with the basic solution and designs a more flexible algorithm for efficient revocation, yet the exposure of sub-keys in private keys makes it highly vulnerable to user-collusion.

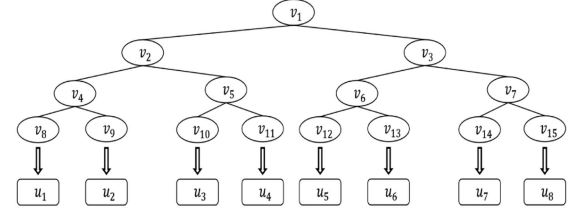


Fig. 2. Schematic diagram of user binary tree construction.

In the TR-CPABE, each user's unique private key is associated with a user property and a user binary tree. These parts are recorded as SK_{attr} and SK_{id} , respectively. Each ciphertext contains ciphertext sub-items related to the minimum covering set, which can be obtained from the revocation list. However, the key vulnerability is that the SK_{attr} and SK_{id} of users' private key are relatively independent. This means that a user can freely combine their own SK_{attr} with other people's SK_{id} to generate a new private key, which leads to significant security risks. For example, if a revoked user combines their holding SK_{attr} with a non-revoked user's SK_{id} , they can successfully restore the decryption ability illegally.

To illustrate this collusion attack, consider the following example. The user binary tree construction is shown in Fig. 2, the revocation list $RL = \emptyset$ controlled by the authorization center and encrypts information according to the access policy "Psychologist and Doctor". Assume that two users, u_1 and u_2 both hold the "Psychologist" attribute and the "Doctor" attribute. The u_1 's SK_{attr} is associated with $\{Psychiatry, Doctor\}$, SK_{id} is associated with the node set $\{v_1, v_2, v_4, v_8\}$ in the associated user binary tree. The u_2 's SK_{attr} is associated with $\{Psychiatry, Doctor\}$, SK_{id} is associated with the node set $\{v_1, v_2, v_4, v_9\}$ in the associated user binary tree. The ciphertext is associated with $\{v_1\}$. At this point, both u_1 and u_2 can decrypt the ciphertext successfully, with both SK_{id} having v_1 . If u_1 leaves the unit, their access authority is revoked, and the private key remains unaffected during this process. However, all ciphertexts will be updated, which means that the associated node set of the ciphertext will be updated from $\{v_1\}$ to $\{v_3, v_5, v_9\}$. There is no non-empty intersection in the u_1 's SK_{id} and ciphertext's associated node set, which deprives the decryption ability of u_1 , while u_2 still maintains their decryption ability using v_9 . In other words, as long as u_1 's SK_{id} can re-fit any point of $\{v_3, v_5, v_9\}$, they will be able to restore their decryption ability. The collusion attack process involves combining u_1 's SK_{attr} with u_2 's SK_{id} to make of u_1 's SK_{id} possess v_9 . This scheme has obvious security flaws in existing-revoked user-collusion due to SK_{attr} and SK_{id} independence in the user private key.

The algorithm construction is somewhat similar to the basic scheme [27]. SK_{id} generation involves the user binary tree secret number—i.e., secret value $X = \{x_i\}_0^{2^{|U|-2}}$ randomly selected by the authorization center for binary tree nodes in the *Setup* stage. Specifically, in the *KeyGen* stage, $SK_{id} = \{x_i\}_{i \in \text{path}(id)}$ is assigned to each user using secret values corresponding to binary tree nodes in the user path set generated by the user ID. In fact, SK_{id} is a subset of secret number X (randomly selected by the authorization center) and has no connection with SK_{attr} generation, forming this relative independence.

D. Contribution

To address the low re-encryption efficiency, inflexible access control, and vulnerability to collusion attacks in existing revocation schemes, we propose UR-CP-ABE enabling direct user revocation. Our contributions are as follows:

- We propose a binary tree-based user revocation mechanism that leverages structural features to optimize user and attribute representation, enabling flexible adjustment of access rights. A user's private key includes sub-items bound to the attribute set and path set, while the ciphertext embeds the access structure and the minimum coverage set derived from the revocation list. Decryption succeeds if the user ID is not in the revocation list and the attribute set satisfies the ciphertext's access policy.
- We develop a flexible revocation algorithm that narrows the re-encryption scope during user revocation to updating only a single sub-item, effectively resolving the high re-encryption cost issue. Specifically, UR-CP-ABE's user-level revocation requires no private key updates, and ciphertext updates only involve a few exponential operations, delivering significant efficiency advantages.
- UR-CP-ABE implements blind protection for key variables, preventing adversaries from deducing secret variables and constructing secret sub-keys, thus mitigating vulnerabilities to collusion attacks.
- UR-CP-ABE supports bidirectional revocation, encompassing both access right revocation and restoration. This design avoids one-way access permission updates and significantly enhances the flexibility of the revocation mechanism.

E. Organization

Section I introduces the research background and motivations. Section II presents the relevant preliminaries. Section III describes the system model. Section IV details the scheme's construction and its correctness analysis. Section V performs the security analysis. Finally, Section VI summarizes the conclusions and highlights the innovations.

II. PRELIMINARIES

A. User Binary Tree

Consider a system with a revocation list RL and user set U . The user binary tree is constructed as follows:

- Leaf nodes correspond one-to-one to users in U (denoted by $userid$). The user binary tree has depth d , accommodating up to $|U| = 2^d$ users and a maximum of $2^{|U|} - 1$ nodes. Each node is assigned a unique index from 1 to $2^{|U|} - 1$ via breadth-first traversal.
- Let $PATH(u)$ denote the user's path set, which is the set of nodes traversed from the leaf node mapped by the user ID to the root node.
- Let $KUNode(RL)$ denote the minimum intersection node set covering all non-revoked users: trace upwards from the leaf nodes of users not in RL to generate this minimum set.
- Due to the user binary tree's characteristics, for $\forall u \notin RL$, there exists a unique node i such that $i \in KUNode(RL) \cap PATH(u)$.

As shown in Fig. 2, if $U = \{u_1, u_2, \dots, u_8\}$ corresponds to leaf nodes $\{v_8, v_9, \dots, v_{15}\}$ in the user binary tree and $RL = \{u_5, u_6\}$, then $PATH(u_3) = PATH(v_{10}) = \{v_1, v_2, v_5, v_{10}\}$ and $KUNode(RL) = \{v_2, v_7\}$, with their intersection being node v_2 .

B. Bilinear Map

Let $\mathbb{Z}_p$ be a finite field. Let G_0, G_1 and G_T be multiplicative cyclic groups of order p , where p is a prime number, then g_0 and g_1 be generators of G_0 and G_1 , respectively. Let $e : G_0 \times G_1 \rightarrow G_T$ be a bilinear map with the following properties.

- Bilinearity: For $\forall u \in G_0, \forall v \in G_1, \forall a, b \in \mathbb{Z}_p$, there is $e(u^a, v^b) = e(u, v)^{ab}$.
- Non-degeneracy: There are at least elements $u \in G_0$ and $v \in G_1$ that satisfy $e(u, v) \neq 1$.
- Computability: For $\forall u \in G_0, v \in G_1$, there exists a polynomial-time algorithm associated with a given security constant ϖ that computes $e(u, v)$ efficiently.

The map $e : G_0 \times G_1 \rightarrow G_T$ is called an asymmetric bilinear map. If $G_0 = G_1$, i.e. the map $e : G_0 \times G_0 \rightarrow G_T$, called a symmetric bilinear map.

C. Access Structure

Let $\{P_1, P_2, \dots, P_n\}$ represent the set of participants, and set $2^P = 2^{\{P_1, P_2, \dots, P_n\}} = \{A | A \subseteq \{P_1, P_2, \dots, P_n\}\}$. The set $\mathbb{A} \subseteq 2^{\{P_1, P_2, \dots, P_n\}}$ is monotonic, for $\forall B, C \subseteq P$, if $B \subseteq C$ and $B \in \mathbb{A}$ implies that $C \in \mathbb{A}$. If $\mathbb{A} \subseteq 2^{\{P_1, P_2, \dots, P_n\}} \setminus \{\emptyset\}$ and monotonic, then $\mathbb{A}$ is an access structure. If $D \in \mathbb{A}$, D is called authorized set, otherwise it's an unauthorized set.

D. Complexity Assumptions

Given a group $\mathbb{G}$ of prime order p generated by g , and randomly chosen elements $f, s, c_1, c_2, \dots, c_q \in \mathbb{Z}_p$, then set

$$\begin{aligned} \vec{g} &= g, g^s, g^f, \dots, g^{(f^q)}, g^{(f^{q+2})}, \dots, g^{(f^{2q})} \\ \forall 1 \leq j \leq q & g^{s/c_j}, g^{f/c_j}, \dots, g^{(f^q/c_j)}, g^{(f^{q+2}/c_j)}, \dots, g^{(f^{2q}/c_j)} \\ \forall 1 \leq j, k \leq q, k \neq j & g^{f \cdot s \cdot c_k / c_j}, \dots, g^{(f^q \cdot s \cdot c_k / c_j)} \end{aligned}$$

The goal is to distinguish the random elements Z in $\mathbb{G}_T$ and $e(g, g)^{f^{q+1}s} \in \mathbb{G}_T$.

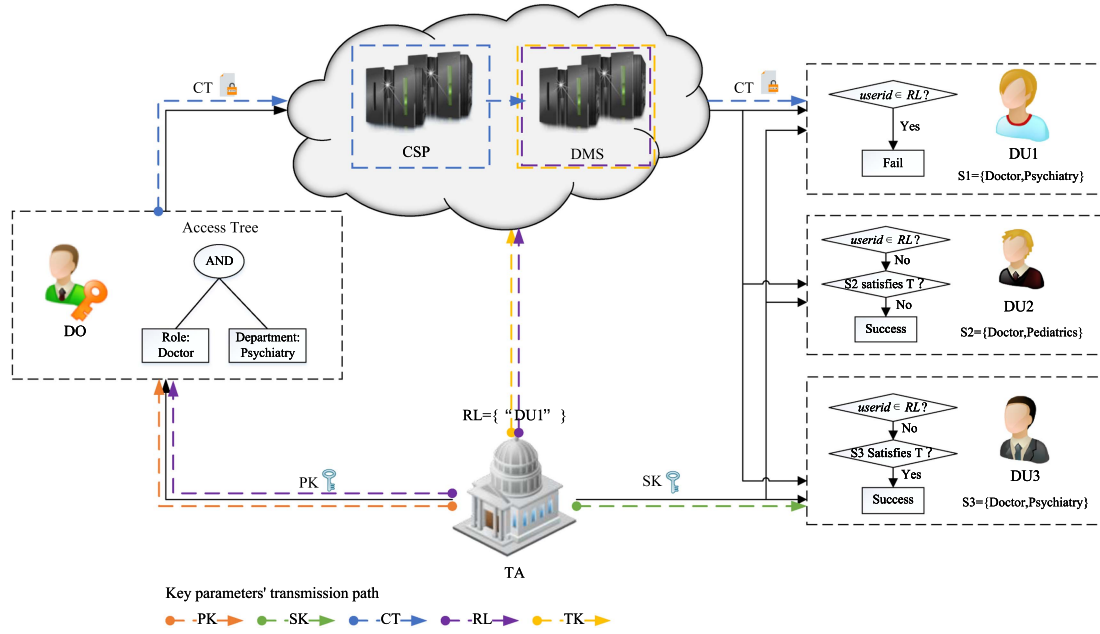


Fig. 3. Framework of UR-CP-ABE.

Algorithm $\mathcal{B}$ outputs $v \in \{0, 1\}$, if satisfied

$$\left| \Pr \left[\mathcal{B}(\vec{Q}, Q = e(g, g)^{f^{q+1}s}) = 0 \right] - \Pr \left[\mathcal{B}(\vec{Q}, Q = Z) = 0 \right] \right| \geq \epsilon$$

Then it has the advantage ϵ in solving the decisional q -parallel BDHE problem in the field $\mathbb{G}$.

Definition 1: If no polynomial-time algorithm has a non-negligible advantage in solving the decisional q -parallel BDHE problems, then the hypothesis holds.

III. SYSTEM MODEL

A. System Framework

Fig. 3 illustrates the entity composition of UR-CP-ABE, comprising five core components.

- **Trusted Authority (TA):** Completely credible and honestly execute the computing tasks assigned by users and outputs correct results;
- **Cloud Service Provider (CSP):** Semi-trusted and responsive to user storage requests, but honest and curious;
- **Data Management Server (DMS):** Semi-reliable, honest re-encryption work, but curious;
- **Data Owner (DO):** Define the access structure, perform encrypted calculations, and upload the shared ciphertext to the cloud server;
- **Data User (DU):** Apply for and receive the user's private key assigned by TA, obtain the shared file from the cloud server and undertake the decryption calculation.

We use color-coded dashed lines to clearly delineate the communication paths of the system's core parameters. Each distinct color corresponds uniquely to one parameter type for easy differentiation: Orange represents the public key transmission path ($TA \rightarrow DO$), green denotes the private key distribution

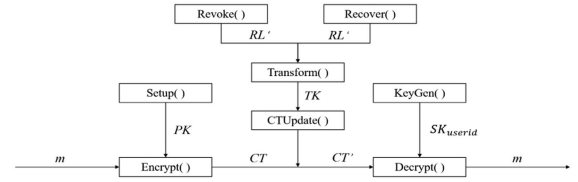


Fig. 4. Algorithm framework of UR-CP-ABE.

route ($TA \rightarrow DU$), and blue illustrates the full lifecycle path of the ciphertext. The ciphertext is generated by DO, uploaded to CSP for secure storage, and then retrieved by DU via download from CSP (i.e., $DO \rightarrow CSP$ for storage, $DU \leftarrow CSP$ for download). Purple identifies the generation and propagation process of RL, which is issued by TA to both DO and DMS. Crucially, DO embeds RL into CT through *Encrypt*, while DMS integrates RL into CT via *CTUpdate* to ensure the seamless and secure subsequent transmission of revocation information, thereby enabling real-time verification of user access rights during ciphertext decryption. Yellow signifies the propagation link of the transformation key ($TA \rightarrow DMS$), which provides technical support for DMS's core functions such as ciphertext update, re-encryption, and access policy adjustment. This color-coding design not only enhances the readability of the system's communication architecture but also facilitates quick tracking and validation of parameter flows during subsequent system testing and verification.

UR-CP-ABE comprises 8 stages (Fig. 4), which shows the logical and data flow relationships between each stage's algorithms, as well as the system's complete operational process and dependency logic.

- **Setup(ϖ, A, B):** Executed by TA, input security parameters ϖ , attribute space A and user binary tree B , output public key PK and master key MSK ;

- *KeyGen*($MSK, userid, S$): Executed by TA , input public key PK , master key MSK , user ID $userid$ and attribute set S , output user's private key SK_{userid} ;
- *Encrypt*($PK, m, (M, \rho), RL$): Executed by DO , input public key PK , plaintext m , access policy (M, ρ) , and revocation list RL , output ciphertext CT ;
- *Decrypt*(SK_{userid}, CT): Executed by DU , input user's private key SK_{userid} and ciphertext CT , output plaintext m ;
- *Revoke*($RL, userid$): Executed by TA , input old revocation list RL and revoked user ID $userid$, output new revocation list RL' ;
- *Transform*(RL, RL'): Executed by TA , input old revocation list RL and new revocation list RL' , output transformation key TK ;
- *CTUpdate*(CT, RL', TK): Executed by DMS , input old ciphertext CT , new revocation list RL' and transformation key TK , output new ciphertext CT' ;
- *Recover*($RL, userid$): Executed by TA , input old revocation list RL and restored user ID $userid$, output new revocation list RL' .

B. Security Model

UR-CP-ABE manages revocations uniquely: ciphertext sub-keys encode both access policies and revocation lists. When a user is revoked, DMS uses the transformation key to update partial ciphertext (sub-ciphertext associated with the minimal coverage set)—the sub-item form remains unchanged, only the associated minimal coverage set and its nodes differ. It updates these sub-items via exponential operations for efficient re-encryption. For security proof, we first use the “**sCPA**” game [27] to describe the security of basic UR-CP-ABE (excluding *Revoke*, *Transform*, *CTUpdate*, and *Recover*). For original and re-encrypted ciphertexts, we use “**sCPA-Or**” and “**sCPA-Up**” games respectively. The framework adopts a selective model (attackers determine the challenge strategy upfront). To extend security to chosen-ciphertext attacks, we propose the **CCA**” game, which builds on **sCPA-Or** and adds adaptive decryption query support.

sCPA: A basic UR-CP-ABE is secure in the selective model if the advantage of an adversary $\mathcal{A}$ in the following game is negligible.

Init: The adversary $\mathcal{A}$ selects a challenged $LSSS$ access structure (M^*, ρ^*) .

Setup: The simulator $\mathcal{B}$ runs *Setup* to get PK , and sends PK to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ repeatedly requests key queries associated with the attribute set $S_i (i = 1, 2, \dots, q)$.

- If $S_i \models (M^*, \rho^*)$, then terminate;
- If $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ generates a key SK_i about the attribute set S_i to $\mathcal{A}$.

Challenge: After $\mathcal{A}$ makes sure Phase 1 is over, it submits two messages of equal length M_0 and M_1 to $\mathcal{B}$. $\mathcal{B}$ randomly selects $\beta \in \{0, 1\}$ and generates ciphertext CT^* by *Encrypt*, which includes M_β associated with (M^*, ρ^*) . Then, $\mathcal{B}$ sends CT^* to $\mathcal{A}$.

Phase 2: Phase 2 is the same as Phase 1.

Guess: $\mathcal{A}$ outputs a guess β' for the random value β . Only when $\beta' = \beta$, He wins the game.

$\mathcal{A}$'s advantage is as follows:

$$Adv_{UR-CP-ABE}^{sCPA}(\mathcal{A}) = \left| Pr[\beta' = \beta] - \frac{1}{2} \right|$$

sCPA-Or: In the selective model, a UR-CP-ABE is considered secure with respect to the original ciphertext when the advantage that an adversary $\mathcal{A}$ has in the following game is so small as to be negligible.

Init: Init here is very similar to Init in **sCPA**. Moreover, $\mathcal{A}$ must select a challenged revocation list RL^* in addition.

Setup: Setup is completely identical to **sCPA**'s Setup.

Phase 1: $\mathcal{A}$ repeatedly requests key queries associated with the user and attribute set $(u_1, S_1), (u_2, S_2), \dots, (u_q, S_q)$.

- If $u_i \in RL^*$, terminate;
- If $u_i \notin RL^*$ and $S_i \models (M^*, \rho^*)$, then terminate;
- If $u_i \notin RL^*$ and $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ generates a key SK_i about (u_i, S_i) to $\mathcal{A}$.

Challenge: There is a great similarity between Challenge and **sCPA**'s Challenge. The difference lies in that $\mathcal{B}$ generates ciphertext CT^* by *Encrypt*, which includes M_β associated with (M^*, ρ^*) and revocation list RL^* .

Phase 2: Phase 2 is the same as Phase 1.

Guess: Guess and the advantages of $\mathcal{A}$ are consistent with those defined in the **sCPA**.

sCPA-Up: In the selective model, if the advantage of an adversary $\mathcal{A}$ in the following game is negligible, then a UR-CP-ABE is secure with regard to the re-encrypted ciphertext.

Init: Init and Init in **sCPA** are remarkably alike. In addition to that, $\mathcal{A}$ also needs to select a challenged revocation list $RL^* (RL \rightarrow RL^*)$.

Setup: Setup is entirely consistent with **sCPA**'s Setup.

Phase 1: $\mathcal{A}$ repeatedly requests key queries associated with the user and attribute set $(u_1, S_1), (u_2, S_2), \dots, (u_q, S_q)$.

- If $u_i \in RL$, terminate;
- If $u_i \notin RL$ and $u_i \notin RL^*$ and $S_i \models (M^*, \rho^*)$, then terminate;
- If $u_i \notin RL$ and $u_i \in RL^*$ and $S_i \models (M^*, \rho^*)$, or $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ generates a key SK_i about (u_i, S_i) to $\mathcal{A}$.

Challenge: Challenge and **sCPA**'s Challenge are very similar. The difference is manifested in the fact that $\mathcal{B}$ generates ciphertext CT^* by *Encrypt*, which includes M_β associated with (M^*, ρ^*) and revocation list RL . Then, $\mathcal{B}$ obtains TK by *Transform*. $\mathcal{B}$ uses TK and RL^* in *CTUpdate* to update CT to CT^* . Finally, $\mathcal{B}$ sends CT^* to $\mathcal{A}$.

Phase 2: $\mathcal{A}$ initiates the following repeatable queries.

① key queries $(u_i, S_i) (i = 1, 2, \dots, q)$:

- If $u_i \in RL^*$, terminate;
- If $u_i \notin RL^*$ and $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ generates SK_i to $\mathcal{A}$.

② ciphertext update queries (CT^*, RL^*, RL') :

If $\exists u_i$, s.t. $u_i \in RL^*$ and $u_i \notin RL'$ and $S_i \models (M^*, \rho^*)$, then terminate; Else $\mathcal{B}$ updates CT^* to CT'^* .

Guess: Guess and the advantages of $\mathcal{A}$ and those defined in the **sCPA** are exactly the same.

Definition 2: If the adversary has at most a negligible advantage in polynomial-time in the “**sCPA**”, “**sCPA-Or**”, and “**sCPA-Up**” security games, then UR-CP-ABE is safe.

CCA: The security model extends the **sCPA-Or** (selective model) by enabling adversaries to conduct adaptive decryption queries, with the game flow strictly aligned with UR-CP-ABE's core mechanisms.

Init: Same as **sCPA-Or**.

Setup: Identical to **sCPA-Or**.

Phase 1: $\mathcal{A}$ initiates repeatable queries:

- Private key query: Consistent with **sCPA-Or**;
- Decryption query: Inputs a non-challenge $CT_{cca} = (CT, vk, \sigma)$; $\mathcal{B}$ first verifies σ , then returns m if $userid \notin RL$ and $S \models (M, \rho)$, else $\perp$.

Challenge: Same as **sCPA-Or**. $\mathcal{B}$ selects β , generates challenge ciphertext $CT_{cca}^* = (CT^*, vk^*, \sigma^*)$ (where $CT^* = \text{Encrypt}(PK, \mathcal{M}_\beta, (M^*, \rho^*), RL^*)$)

Phase 2: Same as Phase 1, excluding queries for $S \models (M^*, \rho^*)$ or CT_{cca}^* decryption.

Guess: Same as **sCPA-Or**.

$$\text{Adv}_{UR-CP-ABE}^{CCA}(\mathcal{A}) = \left| \Pr[\beta' = \beta] - \frac{1}{2} \right|.$$

UR-CP-ABE satisfies CCA security if $\text{Adv}_{UR-CP-ABE}^{CCA}(\mathcal{A})$ is negligible for all polynomial-time $\mathcal{A}$.

IV. OUR CONSTRUCTION

A. UR-CP-ABE

1) **Setup**(ϖ, A, B) $\rightarrow (PK, MSK)$: It selects bilinear groups G_0 and G_T of prime p , denotes G_0 's generator as g , and defines bilinear map $e : G_0 \times G_0 \rightarrow G_T$. TA creates user binary tree B (initial $RL = \emptyset$) based on system users: leaf nodes map one-to-one to U (via $userid$), B has depth d , maximum users $|U| = 2^d$, and total nodes $2|U| - 1$. Random select $a, \alpha \in \mathbb{Z}_p$, $\eta_x \in G_0$ for each attribute $x \in A$ (attribute space), and $N = (n_i)_{i=1}^{2|U|-1} \in \mathbb{Z}_p$ for each node in B , and calculate $T = (t_i | t_i = g^{n_i})_{i=1}^{2|U|-1}$.

$$PK = (g, g^a, e(g, g)^\alpha, \{\eta_x\}_{x \in A}, T = (t_i | t_i = g^{n_i})_{i=1}^{2|U|-1})$$

$$MSK = (\alpha, a, N = (n_i)_{i=1}^{2|U|-1})$$

2) **Encrypt**($PK, m, (M, \rho), RL$) $\rightarrow CT$: Randomly select $s \in \mathbb{Z}_p$, let M be a matrix of $l \times n$, and select random vector $\vec{\varsigma} = (s, \theta_2, \theta_3, \dots, \theta_n) \in \mathbb{Z}_p^n$ to distribute encryption index s . For $i = 1, \dots, l$ compute $\varpi_i = \vec{\varsigma} \cdot M_i$ (M_i is M 's i -th row vector). Randomly select $r_1, \dots, r_l \in \mathbb{Z}_p$, then calculate the ciphertext sub-items of the associated access structure as follows:

$$C = m \cdot e(g, g)^{\alpha s}, C' = g^s,$$

$$(C_1 = g^{a\varpi_1} \eta_{\rho(1)}^{-r_1}, D_1 = g^{r_1}), \dots, (C_l = g^{a\varpi_l} \eta_{\rho(l)}^{-r_l}, D_l = g^{r_l}).$$

Let $KUNode(RL)$ be the minimum covering set of non-revoked users based on the user binary tree, calculated by TA from RL and sent to DO . The ciphertext sub-item for this set is calculated as follows:

$$\{E_j = t_j^s\}_{j \in KUNode(RL)}.$$

Combining the above sub-items, CT is

$$(C, C', \{C_i, D_i\}_{i \in [1, l]}, \{E_j\}_{j \in KUNode(RL)}, (M, \rho), RL).$$

3) **KeyGen**($MSK, userid, S$) $\rightarrow SK_{userid}$: Select $r, \xi \in \mathbb{Z}_p$ at random, the sub-key associated with the user's attribute is computed as follows:

$$K = g^{\alpha + ar} g^{a\xi r}, L = g^{\xi r}, L' = g^\xi, \{K_x = \eta_x^{\xi r}\}_{x \in S}.$$

Let $PATH(userid)$ represent the user path set based on the user binary tree about $userid$, and calculate the key sub-items of the associated user path set as follows:

$$\{F_j = g^{ar} t_j^\xi\}_{j \in PATH(userid)}.$$

Combining the above sub-items, SK_{userid} is

$$(K, L, L', \{K_x\}_{x \in S}, \{F_j\}_{j \in PATH(userid)}).$$

4) **Decrypt**(SK_{userid}, CT) $\rightarrow m$: Assuming that $userid \in RL$ or S does not satisfy (M, ρ) in the ciphertext, returns $\perp$.

Assuming $userid \notin RL$ and S satisfy (M, ρ) in the ciphertext, the calculation is as follows:

First calculate K_1

$$K_1 = e(K, C') = e(g, g)^{\alpha s} e(g, g)^{ars} e(g, g)^{a\xi rs}.$$

Calculate $k \in KUNode(RL) \cap PATH(userid)$, the node k is the intersection of the minimum cover set $KUNode(RL)$ of RL in the ciphertext and the user path set $PATH(userid)$ of $userid$ in the key. Calculated K_2 as follows:

$$K_2 = \frac{e(C', F_k)}{e(E_k, L')} = e(g, g)^{ars}.$$

Suppose $I = \{i : \rho(i) \in S\} \subseteq \{1, 2, \dots, l\}$, and $\{\chi_i \in \mathbb{Z}_p\}_{i \in I}$ is a set of constants. If $\{\varpi_i\}$ is a valid secret share for any secret s above M , then $\sum_{i \in I} \chi_i \varpi_i = s$ (the value χ_i that satisfies this requirement is not unique). K_3 can be calculated according to $\{\chi_i\}_{i \in I}$

$$K_3 = \prod_{i \in I} (e(C_i, L) e(D_i, K_{\rho(i)}))^{\chi_i} = e(g, g)^{a\xi rs}.$$

Combining the above, the final calculation process is

$$C / \frac{K_1}{K_2 \cdot K_3} = m.$$

In this way, the plaintext message m can be recovered.

B. User Revoke

1) **Revoke**($RL, userid$) $\rightarrow RL'$:

$$RL' = RL \cup \{userid\}$$

TA outputs the updated revocation list RL' .

2) **Transform**(RL, RL') $\rightarrow TK$: Let $KUNode(RL')$ denote the minimum coverage set of RL' , $KUNode(RL)$ denote the minimum coverage set of RL , and $e = KUNode(RL) - KUNode(RL')$, $e' = KUNode(RL') - KUNode(RL)$.

For all $node1 \in e$, $node2 \in e'$, let $index(node)$ represent the index value of $node$ in B , calculation steps are as follows:

- Exist $\delta, \delta' \in N^*$ satisfy $index(node2) = 2^\delta \cdot index(node1) + \delta'$ ($\delta' \in [1, 2^\delta - 1]$), then calculate $TK \cup \{n_{index(node2)/n_{index(node1)}}\}$.
- Exist $\delta, \delta' \in N^*$ satisfy $index(node1) = 2^\delta \cdot index(node2) + \delta'$ ($\delta' \in [1, 2^\delta - 1]$), then calculate $TK \cup \{n_{index(node2)/n_{index(node1)}}\}$.

After generating TK and outputting updated RL' , TA performs verifiability-enhanced secure transmission processing. Note that the algorithm's core logic remains standard TK and RL' parameter passing (as described), while TP is just a practical measure to enhance real-world transmission security.

TA generates a UTC millisecond-precision TS via a trusted time source, concatenates TS to RL' to form $RL'' = RL' || TS$, computes

$$Sig = Sign_a(H(TK || RL'')) = e(g, (H(TK || RL''))^a)$$

where $H : \{0, 1\}^* \rightarrow G_0$. Finally, TA encapsulates TK , RL'' , and Sig into TP , encrypts TP with PK , and transmits it to DMS .

3) $CTUupdate(CT, RL', TK) \rightarrow CT'$: Upon receiving the transmission package TP , DMS verifies securely before ciphertext update: decrypts TP (with TA 's pre-distributed *Setup* key) to extract TK , RL'' , and Sig ; separates timestamp TS from RL'' (discards TP and alerts TA if time difference exceeds threshold, else proceeds); verifies Sig against $H(TK || RL'')$ using TA 's g^a , as shown in the equation below:

$$Sig \stackrel{?}{=} e(g^a, H(TK || RL''))$$

discards TP on failure, or strips TS from RL'' to restore RL' and run original $CTUupdate$ on success.

According to $KUNode(RL)$ and $KUNode(RL')$, the update process for any $j' \in KUNode(RL')$ is as follows.

If $j' \in KUNode(RL') \cap KUNode(RL)$, then $j = j'$ for some $j \in KUNode(RL)$, the sub-item in CT' is

$$\tilde{E}_{j'} = t_j^s = t_j^s.$$

When $j' \notin KUNode(RL') \cap KUNode(RL)$, exist $n_{index(j')}/n_{index(j)} \in TK$ and $j \in KUNode(RL)$

$$\tilde{E}_{j'} = (t_j^s)^{n_{index(j')}/n_{index(j)}} = t_{j'}^s.$$

Calculate the sub-item of the associated access structure.

$$\tilde{C} = m \cdot e(g, g)^{\alpha s}, \tilde{C}' = g^s,$$

$$(\tilde{C}_1 = g^{a\omega_1} \eta_{\rho(1)}^{-r_1}, \tilde{D}_1 = g^{r_1}), \dots, (\tilde{C}_l = g^{a\omega_l} \eta_{\rho(l)}^{-r_l}, \tilde{D}_l = g^{r_l})$$

Combining the above and other sub-items to form CT' .

$$(\tilde{C}, \tilde{C}', \{\tilde{C}_i, \tilde{D}_i\}_{i \in [1, l]}, \{\tilde{E}_{j'}\}_{j' \in KUNode(RL')}, (M, \rho), RL')$$

DMS uploads CT' to the cloud server to replace CT .

C. User Recovery

1) $Recover(RL, userid) \rightarrow RL'$:

$$RL' = RL - \{userid\}$$

TA outputs the updated revocation list RL' . Other algorithms involved in user recovery, as algorithms described in *Transform* and *CTUupdate*.

D. Correctness Analysis

If the user's attribute set satisfies the access policy of the CT , and the user has not been revoked, then S satisfies (M, ρ) , $\{F_k\}$ matches $\{E_k\}$, and the DU can decrypt K_2 and K_3 .

$$\begin{aligned} K_2 &= \frac{e(C', F_k)}{e(E_k, L')} = \frac{e(g^s, g^{ar} t_k^\xi)}{e(t_k^s, g^\xi)} \\ &= \frac{e(g^s, g^{ar}) e(g^s, t_k^\xi)}{e(t_k^s, g^\xi)} \\ &= \frac{e(g, g)^{sar} e(g, t_k)^{s\xi}}{e(t_k, g)^{s\xi}} \\ &= e(g, g)^{ars} \end{aligned}$$

$$\begin{aligned} K_3 &= \prod_{i \in I} (e(C_i, L) e(D_i, K_{\rho(i)}))^{\chi_i} \\ &= \prod_{i \in I} \left(e(g^{a\omega_i} \eta_{\rho(i)}^{-r_i}, g^{\xi r}) e(g^{r_i}, \eta_{\rho(i)}^{\xi r}) \right)^{\chi_i} \\ &= \prod_{i \in I} \left(e(g^{a\omega_i}, g^{\xi r}) e(\eta_{\rho(i)}^{-r_i}, g^{\xi r}) e(g^{r_i}, \eta_{\rho(i)}^{\xi r}) \right)^{\chi_i} \\ &= \prod_{i \in I} \left(e(g, g)^{a\omega_i \xi r} e(\eta_{\rho(i)}, g)^{-r_i \xi r} e(g, \eta_{\rho(i)})^{r_i \xi r} \right)^{\chi_i} \\ &= \prod_{i \in I} e(g, g)^{a\omega_i \chi_i \xi r} = e(g, g)^{a\xi r s} \end{aligned}$$

The message m can be recovered as:

$$C / \frac{K_1}{K_2 \cdot K_3} = \frac{m \cdot e(g, g)^{\alpha s}}{\frac{e(g, g)^{\alpha s} \cdot e(g, g)^{ars} \cdot e(g, g)^{a\xi r s}}{e(g, g)^{ars} \cdot e(g, g)^{a\xi r s}}} = m$$

When the user is revoked, RL is updated to RL' by *User revoke*, then E_j is updated to $\tilde{E}_{j'}$ by *CTUupdate*. At this time, there is no common nodes between $PATH(userid)$ and $KUNode(RL')$. F_j can no longer match any of the items in the $\tilde{E}_{j'}$. So even if S satisfies (M, ρ) , the user cannot successfully compute K_2 .

An example of binary tree changes during user revocation illustrates the revocation process's correctness (Fig. 5). Before the users u_4 and u_5 lose access rights, RL is $\{u_7, u_8\}$ and $KUNode(RL)$ is $\{v_2, v_6\}$. It is evident that u_4 can compute K_2 through the common node v_2 of $PATH(u_4)$ and $KUNode(RL)$. Similarly, u_5 can be achieved through node v_6 . After the revocation, RL changes to $RL' = \{u_4, u_5, u_7, u_8\}$ and $KUNode(RL')$ is $\{v_4, v_{10}, v_{13}\}$. Neither $PATH(u_4)$ nor $PATH(u_5)$ has any common nodes with $KUNode(RL')$ anymore. In this example, the process for generating TK in the *Transform* algorithm is as follows:

$$e = KUNode(RL) - KUNode(RL') = \{v_2, v_6\}$$

$$e' = KUNode(RL') - KUNode(RL) = \{v_4, v_{10}, v_{13}\}$$

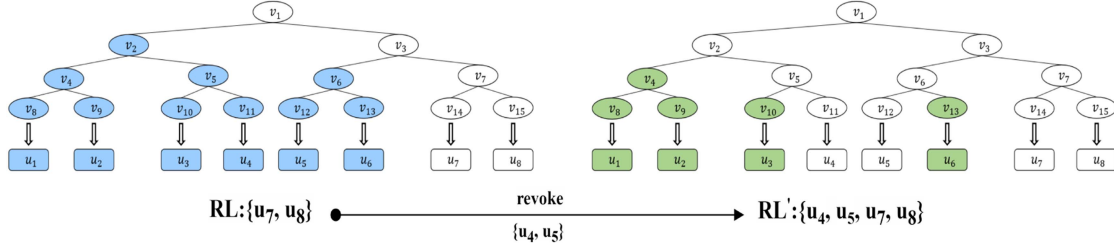


Fig. 5. Instance diagram of user revocation binary tree.

For all $node1 \in e$, $node2 \in e'$, identify the parent-child relationship to determine TK .

$$index(v_4) = 2^1 \cdot index(v_2), TK \cup \{n_4/n_2\}$$

$$index(v_{10}) = 2^2 \cdot index(v_2) + 2, TK \cup \{n_{10}/n_2\}$$

$$index(v_{13}) = 2^1 \cdot index(v_6) + 1, TK \cup \{n_{13}/n_6\}$$

It is noteworthy that there is no δ and δ' between v_{13} and v_2 that allows them to satisfy the ancestor-child relationship. The approach for *User recovery* and *User revoke* is fundamentally the same.

E. User Binary Tree Update and Synchronization

This section details how changes to the user binary-tree structure are handled and synchronized with the *DMS*, enabling flexible evolution while preserving compatibility with existing user permissions and maintaining system efficiency during adjustments.

- When user count exceeds the original tree's maximum capacity 2^d (dynamic expansion), an extended binary tree with depth $d' \geq d$ is built. The original tree retains all its nodes to maintain the *PATH* binding of existing users, and new users are exclusively mapped to the leaf nodes of the extended tree. The *TA* generates a new node secret set $N' = (n'_i)_{i=1}^{2^{|U'|}-1}$ (where U' denotes the set of new users) and the corresponding public parameter set $T' = (t'_i | t'_i = g^{n'_i})$ for the extended tree, then updates the *PK* by integrating T' into the original binary tree parameter set T . Existing users' *SK* remain unchanged, while new users obtain *SK* bound to the *PATH* of the extended tree through the *KeyGen* algorithm.

- For abnormal node adjustments, the *TA* first locates the abnormal node and its subordinate subtree, then regenerates the secret values only for this subtree (instead of the entire tree) and calculates the key update factor $\Delta n_i = n'_i/n_i$ (where n_i is the original node secret and n'_i is the new secret). The *TA* pushes Δn_i to users whose *PATH* includes the abnormal node; these users locally update the corresponding sub-item as $E_j = E_j^{\Delta n_i}$, eliminating the need for full *SK* regeneration.

To ensure efficiency and security, the synchronization of binary tree changes to the *DMS* adopts an *incremental push + verification confirmation* mechanism. Specifically, after updating the binary tree, the *TA* initiates synchronization by generating a synchronization package containing the update type (expansion or abnormal adjustment), the index set of changed nodes, updated public parameters (T' for expansion,

$\Delta t_i = g^{\Delta n_i}$ for abnormal adjustment), and a subset of *TK* related to changed nodes (for abnormal adjustment scenarios), encrypts the package using *PK*, and transmits it to *DMS*. Upon receiving the package, *DMS* first verifies the *TA*'s digital signature to confirm source legitimacy, then parses the package content to update its local binary tree parameter library, and subsequently updates only the *CT* sub-items associated with the changed nodes via *CTUpdate*. After the update, *DMS* sends a confirmation receipt to *TA*, which records the synchronization log to form a closed-loop process.

This part ensures UR-CP-ABE's logarithmic scalability. As the number of users $|U|$ grows, the binary tree depth d increases logarithmically (satisfying $d = \lceil \log_2 |U| \rceil$). When the user count reaches 100,000, the required binary tree depth is only $d = 17$ ($2^{16} = 65536 < 100000 \leq 2^{17} = 131072$), growing logarithmically with the user scale. The logarithmic growth of tree depth and node count ensures the scheme's time and space complexity do not surge with expanding user scale, meeting the scalability requirements of large-scale distributed systems in practical applications.

V. SECURITY ANALYSIS

A. CPA Security Proof

Theorem 1: Under the decisional q -parallel *BDHE* assumption, the adversary cannot use the selectively chosen plaintext attack to break basic UR-CP-ABE proposed in this paper.

Proof: If the polynomial-time adversary $\mathcal{A}$ can break the basic UR-CP-ABE with advantage ϵ , a simulator $\mathcal{B}$ can be constructed to break the decisional q -parallel *BDHE* hardness problem with advantage $\epsilon/2$. Detailed simulation steps:

The challenger $\mathcal{C}$ generates $\vec{g}$ and randomly choose $v \in \{0, 1\}$, if $v = 0$, $\mathcal{C}$ sets $Q = e(g, g)^{f^{q+1}s}$; otherwise, $\mathcal{C}$ sets $Q = Z$, where Z is chosen randomly from $\mathbb{G}_T$.

Init: The simulator $\mathcal{B}$ takes in the q -parallel *BDHE* challenge $\vec{g}$ and Q from $\mathcal{C}$, $\mathcal{A}$ chooses an access structure (M^*, ρ^*) which will be challenged, where M^* is an $l^* \times n^*$ matrix.

Setup: $\mathcal{B}$ does

(1) Choose $\alpha' \in \mathbb{Z}_p$ such that $e(g, g)^\alpha = e(g^f, g^{f^q}) \cdot e(g, g)^{\alpha'}$, which implicitly sets $\alpha = \alpha' + f^{q+1}$;

(2) Set $g^a = g^f$, which implicitly sets $a = f$;

(3) For each $x \in A$, A is the attribute space, choose a random value $\mu_x \in \mathbb{Z}_p$. Let X be the set of all i such that $\rho^*(i) = x$. Therefore, $\mathcal{B}$ writes η_x

$$\eta_x = g^{\mu_x} \prod_{i \in X} g^{f M_{i,1}^*/c_i} \cdot g^{f^2 M_{i,2}^*/c_i} \dots g^{f^{n^*} M_{i,n^*}^*/c_i}.$$

Note: Parameters are randomly distributed due to g^{μ_x} .

(4) Build a user binary tree. For each $i(1 \leq i \leq 2|U| - 1)$, randomly select $n_i \in \mathbb{Z}_p$, calculate $t_i = g^{n_i}$

$$PK = (g, e(g, g)^\alpha, g^a, \{\eta_x\}_{x \in A}, T = (t_i | t_i = g^{n_i})_{i=1}^{2|U|-1}).$$

Phase 1: The adversary $\mathcal{A}$ submits a series of tuples $(S_1, S_2, \dots, S_q)$ to $\mathcal{B}$ requesting private keys, and $\mathcal{B}$ responds to private key queries.

Case 1: If $S_i \models (M^*, \rho^*)$, then terminate;

Case 2: If $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ operates as follows:

(1) Find a vector $\vec{\chi} = (\chi_1, \chi_2, \dots, \chi_{n^*}) \in \mathbb{Z}_p^{n^*}$, such that $\chi_1 = -1$ and $\vec{\chi} \cdot M_i^* = 0$, which i satisfies $\rho^*(i) \in S$;

(2) Select $r \in \mathbb{Z}_p$ and $\beta \in \mathbb{Z}_p$ randomly, and implicitly set $\xi = \frac{1}{r}(\beta + \chi_1 f^q + \chi_2 f^{q-1} + \dots + \chi_{n^*} f^{q-n^*+1})$;

(3) Calculate K, L, L' as follows:

$$K = g^{\alpha'} g^{a(r+\beta)} \prod_{i=2, \dots, n^*} (g^{f^{q+2-i}})^{\chi_i} = g^{\alpha+ar} g^{a\xi r}$$

$$L = g^\beta \left[\prod_{i=1, \dots, n^*} (g^{f^{q+1-i}})^{\chi_i} \right] = g^{\xi r}$$

$$L' = g^{\frac{\beta}{r}} \left[\prod_{i=1, \dots, n^*} (g^{f^{q+1-i}})^{\chi_i} \right]^{\frac{1}{r}} = g^\xi$$

(4) For $x \in S$ compute K_x . If none of i satisfy $\rho^*(i) = x$, set $K_x = L^{\mu_x}$. When $i \in X$ enabled $\rho^*(i) = x$, K_x is

$$K_x = L^{\mu_x} \prod_{i \in X} \prod_{j=1, \dots, n^*} \left[\left(g^{(f^j/c_i)} \right)^\beta \prod_{k=1, \dots, n^*, k \neq j} \left(g^{(f^{q+j-k+1}/c_i)} \right)^{\chi_k} \right]^{M_{i,j}^*} = \eta_x^{\xi r} \left\{ F_j = g^{ar} (L')^{n_j} = g^{ar} (g^\xi)^{n_j} = g^{ar} t_j^\xi \right\}_{j \in \text{PATH}(u_i)}.$$

Challenge: Finally create the challenge ciphertext. $\mathcal{A}$ sends two messages $\mathcal{M}_0$ and $\mathcal{M}_1$ of equal length to $\mathcal{B}$ as well as RL^* . $\mathcal{B}$ operates as follows:

1) Choose $\beta \in \{0, 1\}$ at random. Create C and C'

$$C = \mathcal{M}_\beta \cdot Q \cdot e(g^s, g^{\alpha'}), C' = g^s.$$

2) $\mathcal{B}$ randomly selects $\theta'_2, \dots, \theta'_{n^*} \in \mathbb{Z}_p^*$, and uses the following vector to share the secret

$$\vec{s} = (s, sf + \theta'_2, sf^2 + \theta'_3, \dots, sf^{n^*-1} + \theta'_{n^*}) \in \mathbb{Z}_p^{n^*}.$$

3) Random select $r'_1, \dots, r'_{l^*} \in \mathbb{Z}_p^*$. R_i is defined as the set of $k \neq i$, which satisfy $\rho^*(i) = \rho^*(k)$. And the challenge ciphertext sub-keys are

$$C_i = \eta_{\rho^*(i)}^{r'_i} (g^{sc_i})^{-\mu_{\rho^*(i)}} \left(\prod_{j=2, \dots, n^*} (g^f)^{M_{i,j}^* \theta'_j} \right) \left(\prod_{k \in R_i} \prod_{j=1, \dots, n^*} (g^{f^j s(c_i/c_k)})^{-M_{k,j}^*} \right), i = 1, \dots, l^*$$

$$D_i = g^{-r'_i} g^{sc_i}.$$

4) For $j \in KUNode(RL^*)$, $\mathcal{B}$ calculates

$$E_j = (g^s)^{n_j} = t_j^s$$

$$CT = (C, C', \{\forall i \in [1, l^*] : C_i, D_i\}, \{E_j\}_{j \in KUNode(RL^*)}).$$

Phase 2: Phase 2 is the same as Phase 1.

Guess: β' is the guess of $\mathcal{A}$'s final output. If $\beta' = \beta$, $\mathcal{B}$ finally outputs $v' = 0$, this means the guess $Q = e(g, g)^{f^{q+1} \cdot s}$; otherwise, $\mathcal{B}$ outputs $v' = 1$, this indicates guessing that Q is a random group element in $\mathbb{G}_T$.

When $v = 1$, $\mathcal{A}$ gains no information about β . Therefore, we have $Pr(\beta \neq \beta' | v = 1) = \frac{1}{2}$. Since $\mathcal{B}$ guesses $v' = 1$ when $\beta \neq \beta'$, we have $Pr(v' = v | v = 1) = \frac{1}{2}$.

When $v = 0$, if $\mathcal{A}$'s advantage in breaking UR-CP-ABE is ϵ , then $Pr(\beta = \beta' | v = 0) = \frac{1}{2} + \epsilon$. Since $\mathcal{B}$ guesses $v' = 0$ when $\beta = \beta'$, we get $Pr(v' = v | v = 0) = \frac{1}{2} + \epsilon$.

Therefore, when $\mathcal{A}$ breaks through UR-CP-ABE with an advantage of ϵ , it can solve the decisional q -parallel BDHE problems with an advantage of $\frac{1}{2} Pr[v' = v | v = 0] + \frac{1}{2} Pr[v' = v | v = 1] - \frac{1}{2} = \frac{1}{2}(\frac{1}{2} + \epsilon) + \frac{1}{2}\frac{1}{2} - \frac{1}{2} = \frac{1}{2}\epsilon$.

Theorem 2: Under the decisional q -parallel BDHE assumption, no adversary can compromise the original ciphertext of UR-CP-ABE via a selectively chosen plaintext attack.

Proof: Suppose the polynomial-time adversary $\mathcal{A}$ can break UR-CP-ABE at the original ciphertext with advantage ϵ . We can then construct a simulator $\mathcal{B}$ to solve the decisional q -parallel BDHE hardness problem with advantage $\epsilon/2$. Detailed simulation process below:

Init: Init here is similar to that in **sCPA**, with $\mathcal{A}$ additionally selecting a challenged revocation list RL^* .

Setup: $\mathcal{B}$ obtains the public key PK from the challenger $\mathcal{C}$ (simulating basic UR-CP-ABE) and gives it to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ submits $(u_1, S_1), (u_2, S_2), \dots, (u_q, S_q)$ to $\mathcal{B}$ requesting private keys, and $\mathcal{B}$ responds to private key queries.

Case 1: If $u_i \in RL^*$, terminate;

Case 2: If $u_i \notin RL^*$ and $S_i \models (M^*, \rho^*)$, terminate;

Case 3: If $u_i \notin RL^*$ and $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ sends (u_i, S_i) to $\mathcal{C}$, and $\mathcal{C}$ generates a key SK_i by $\text{KeyGen}(MSK, u_i, S_i)$ to $\mathcal{B}$. Then $\mathcal{B}$ sends the SK_i to $\mathcal{A}$.

Challenge: There is a great similarity between Challenge and **sCPA**'s Challenge. The difference lies in that $\mathcal{B}$ chooses $\beta \in \{0, 1\}$ at random and sends $\mathcal{M}_\beta$ to $\mathcal{C}$, and $\mathcal{C}$ generates a ciphertext CT^* by $\text{Encrypt}(PK, \mathcal{M}_\beta, (M^*, \rho^*), RL^*)$ to $\mathcal{B}$. Then $\mathcal{B}$ sends the CT^* to $\mathcal{A}$.

Phase 2: Phase 2 is the same as Phase 1.

Guess: $\mathcal{A}$ outputs a guess of β' . $\mathcal{B}$ outputs $v' = 0$, when $\beta' = \beta$; otherwise, $\mathcal{B}$ outputs $v' = 1$.

Thus, if $\mathcal{A}$ breaks UR-CP-ABE at the original ciphertext with advantage ϵ , it also breaks the basic UR-CP-ABE with the same advantage ϵ , and can solve the decisional q -parallel BDHE problem with advantage $\frac{1}{2}\epsilon$.

Theorem 3: With the decisional q -parallel BDHE assumption and the discrete logarithm (DL) problem in place, an adversary cannot break the re-encrypted ciphertext of UR-CP-ABE via a selectively chosen plaintext attack.

Proof: Suppose the polynomial-time adversary $\mathcal{A}$ breaks UR-CP-ABE at the re-encrypted ciphertext with advantage ϵ . We can construct a simulator $\mathcal{B}$ to solve either the decisional q -parallel BDHE hardness problem or the discrete logarithm (DL) problem with advantage $\epsilon/2$. The DL problem's probability calculation is analogous to the aforementioned q -parallel BDHE problem. Detailed simulation process below:

Init: Init in **sCPA** is nearly identical. Additionally, $\mathcal{A}$ selects a challenged revocation list $RL^*(RL \rightarrow RL^*)$.

Setup: $\mathcal{B}$ obtains PK from the challenger $\mathcal{C}$ (simulating basic UR-CP-ABE) and gives it to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ submits $(u_i, S_i), i = 1, 2, \dots, q$ to $\mathcal{B}$ requesting private keys, and $\mathcal{B}$ responds to private key queries.

Case 1: If $u_i \in RL$, terminate;

Case 2: If $u_i \notin RL$ and $u_i \notin RL^*$ and $S_i \models (M^*, \rho^*)$, then terminate;

Case 3: If $u_i \notin RL$ and $u_i \in RL^*$ and $S_i \models (M^*, \rho^*)$, or $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ sends (u_i, S_i) to $\mathcal{C}$, and $\mathcal{C}$ generates SK_i by $KeyGen(MSK, u_i, S_i)$ to $\mathcal{B}$ (If u_i has already been queried, it can be directly retrieved from the private table and returned). Then $\mathcal{B}$ sends the SK_i to $\mathcal{A}$.

Challenge: $\mathcal{A}$ sends $\mathcal{M}_0$ and $\mathcal{M}_1$ of equal length to $\mathcal{B}$. $\mathcal{B}$ chooses $\beta \in \{0, 1\}$ at random and sends $\mathcal{M}_\beta$ to $\mathcal{C}$, and $\mathcal{C}$ generates a ciphertext CT by $Encrypt(PK, \mathcal{M}_\beta, (M^*, \rho^*), RL)$ and a key TK by $Transform(RL, RL^*)$ to $\mathcal{B}$. $\mathcal{B}$ generates CT^* by $CTUpdate(CT, TK, RL^*)$ and sends it to $\mathcal{A}$.

Phase 2: $\mathcal{A}$ initiates the following repeatable queries.

① key queries $(u_i, S_i)(i = 1, 2, \dots, q)$:

Case 1: If $u_i \in RL^*$, terminate;

Case 2: If $u_i \notin RL^*$ and $S_i \not\models (M^*, \rho^*)$, $\mathcal{B}$ sends (u_i, S_i) to $\mathcal{C}$, and $\mathcal{C}$ generates SK_i by $KeyGen(MSK, u_i, S_i)$ to $\mathcal{B}$. Then $\mathcal{B}$ sends the SK_i to $\mathcal{A}$.

② ciphertext update queries (CT^*, RL^*, RL') :

If $\exists u_i$, s.t. $u_i \in RL^*$ and $u_i \notin RL'$ and $S_i \models (M^*, \rho^*)$, terminate; Else $\mathcal{B}$ sends (RL^*, RL') to $\mathcal{C}$, and $\mathcal{C}$ generates TK by $Transform(RL^*, RL')$ to $\mathcal{B}$. $\mathcal{B}$ generates CT' by $CTUpdate(CT^*, TK, RL')$ and sends the CT' to $\mathcal{A}$.

Guess: β' is the guess of $\mathcal{A}$'s final output. If $\beta' = \beta$, $\mathcal{B}$ finally outputs $v' = 0$; otherwise, $\mathcal{B}$ outputs $v' = 1$.

Therefore, for $\forall S_i \not\models (M^*, \rho^*)$, when $\mathcal{A}$ breaks through UR-CP-ABE at re-encrypted ciphertext with an advantage of ϵ , and it can break through basic UR-CP-ABE with an advantage of ϵ , and it also can solve the decisional q -parallel BDHE problems with an advantage of $\frac{1}{2}\epsilon$; For $\exists S_i \models (M^*, \rho^*)$ and $u_i \in RL^*$, when $\mathcal{A}$ breaks through UR-CP-ABE at re-encrypted ciphertext with an advantage of ϵ , and it can break through $(g^{n_k})^x = g^{n_j}$ under $k \in PATH(u_i)$ and $k \notin KUNode(RL^*)$ and $j \in KUNode(RL^*)$, and it also can solve the discrete logarithm (DL) problem.

B. CCA Security Proof

This section extends the security guarantee to the chosen-ciphertext attack (CCA) scenario using the Canetti-Halevi-Katz (CHK) standard transformation framework [28], compatible with the selective security model (**sCPA**, **sCPA-Or**, **sCPA-Up**) without modifying core algorithms.

To achieve CCA security, we introduce a strongly existentially unforgeable Lamport one-time signature scheme $\Sigma = (SigSetup, SigKeyGen, Sign, Verify)$. The transformations only adjusts the ciphertext format and decryption pre-processing without modifying core algorithms:

- Ciphertext enhancement: When the DO generates the core ciphertext CT via the $Encrypt$, it uses $SigKeyGen$ to obtain a signature key pair (vk, sk) (one pair per ciphertext, satisfying the one-time requirement), signs CT 's core verifiable components $(C', \{E_j\}, (M, \rho))$ via $Sign(sk, CT)$ to get σ . Embeds vk into the final ciphertext $CT_{cca} = (CT, vk, \sigma)$ (ensuring the verifier can obtain vk to verify the signature);
- Decryption verification: when the DU or DMS receives CT_{cca} , it first runs $Verify(vk, CT, \sigma)$ —returns $\perp$ if verification fails (indicating forged or tampered ciphertext), otherwise executes the $Decrypt$ on CT .

Theorem 4: If UR-CP-ABE satisfies CPA security (including **sCPA**, **sCPA-Or**, **sCPA-Up**) under the decisional q -parallel BDHE assumption, and Σ is strongly existentially unforgeable, then the extended scheme satisfies CCA security.

Proof: A reduction method is adopted. Assume there exists a polynomial-time adversary $\mathcal{A}$ that breaks CCA security with non-negligible $Adv_{UR-CP-ABE}^{CCA}(\mathcal{A})$; we construct a simulator $\mathcal{B}$ to break the UR-CP-ABE's CPA security.

Init: $\mathcal{B}$ receives the CPA challenger's decisional q -parallel BDHE challenge $\vec{Q}$ and random element Q , runs $SigSetup$ and $SigKeyGen$ to generate (vk^*, sk^*) , constructs the CCA game public key $PK_{cca} = (PK, vk^*)$ (where PK is generated via $\vec{Q}$ as in the original CPA proof). $\mathcal{B}$ sends PK_{cca} to $\mathcal{A}$.

Phase 1: For $\mathcal{A}$'s queries:

- Private key query: $\mathcal{B}$ simulates the $KeyGen$ algorithm to generate SK_{userid} (follows the original CPA proof constraints, terminates if $S \models (M^*, \rho^*)$) and returns it to $\mathcal{A}$;
- Decryption query: For $CT_{cca} = (CT, vk, \sigma)$, $\mathcal{B}$ verifies σ via $Verify(vk, CT, \sigma)$: returns $\perp$ if $vk \neq vk^*$ (since Σ is one-time, $\mathcal{A}$ cannot obtain the corresponding sk) or verification fails; if verification passes, $\mathcal{B}$ extract the attribute set S and userid from CT and the private key query history, send a private key query for $(userid, S)$ to the CPA challenger (complying with **sCPA** constraints), and use the returned SK_{userid} to run $Decrypt$ and return the result.

Challenge: $\mathcal{A}$ submits $\mathcal{M}_0, \mathcal{M}_1$ and (M^*, ρ^*) (with RL^*). $\mathcal{B}$ requests the CPA challenger to generate $CT^* = Encrypt(PK, \mathcal{M}_\beta, (M^*, \rho^*), RL^*)$, signs CT^* with sk^* to get σ^* , and sends $CT_{cca}^* = (CT^*, vk^*, \sigma^*)$ to $\mathcal{A}$.

Phase2: Consistent with Phase 1, except $\mathcal{A}$ cannot request private keys for attribute sets satisfying $S \models (M^*, \rho^*)$, nor submit CT_{cca}^* for decryption.

Guess: Same as the Guess phase in **sCPA-Or**.

Since Σ is strongly existentially unforgeable, $\mathcal{A}$ cannot construct an invalid ciphertext with a valid signature corresponding to vk^* . Thus, $Adv_{UR-CP-ABE}^{sCPA}(\mathcal{B}) = Adv_{UR-CP-ABE}^{CCA}(\mathcal{A})$, which contradicts the assumption that UR-CP-ABE satisfies CPA security under the decisional q -parallel BDHE assumption. Therefore, Theorem 4 holds.

TABLE II
SYMBOL DEFINITION

Symbol	Definition
E_{G_0}	Modular exponentiation in G_0
E_{G_T}	Modular exponentiation in G_T
E_p	Bilinear pairing operation
E_k	Symmetric encryption
D_k	Symmetric decryption
L_{G_0}	Bit-length of field element in G_0
L_{G_T}	Bit-length of field element in G_T
$L_{\mathbb{Z}_p}$	Bit-length of field element in $\mathbb{Z}_p$
$ T $	Minimum number of internal nodes that satisfy the access structure
$ U $	Number of users
$ A $	Size of attribute space
$ S_T $	Number of attributes in the accesses structure
$ S_U $	Number of user's attributes
$ S_R $	Average number of revoked users' attributes
$ S_d $	Number of labels in [26]
$ N_S $	Average number of attribute group's users
$ N_C $	Number of nodes in the minimum covering set
$ N_R $	Number of revoked users

VI. PERFORMANCE ANALYSIS

We select five representative schemes [13], [15], [16], [25], [26] as baselines, and distill effective general algorithms via comparative analysis. These lay the foundation for integrating IoT, fog computing and other technologies into scenario-converged data sharing security solutions.

The performance evaluation of UR-CP-ABE includes theoretical and experimental analysis validates the theoretical results and demonstrates the efficiency of UR-CP-ABE in decryption and reinforcement stages, making it well-suited for file sharing in distributed environments supporting user-level revocation.

A. Theoretical Analysis

The definition of the relevant symbols involved in theoretical analysis is shown in Table II.

1) *Storage Overhead*: UR-CP-ABE shares similar public/master key generation storage overhead with [13]. Unlike [15], [16], [25], [26], it assigns one secret variable to each node of the user binary tree ($T = \{t_i | t_i = g^{n_i}\}_{i=1}^{2|U|-1}$, $N = \{n_i\}_{i=1}^{2|U|-1}$). With $|U|$ leaf nodes, the tree has $2|U| - 1$ total nodes, additional overheads are $(2|U| - 1)L_{G_0}$ and $(2|U| - 1)L_{\mathbb{Z}_p}$.

UR-CP-ABE distributes and revokes key sub-items per user via their user path set during key generation. With a user binary tree depth d (and $d + 1$ path-related sub-items), the base storage cost is $(d + 1)L_{G_0}$. Adding $|S_U|$ attribute sub-items and one public key sub-item, the private key storage cost is $[3 + (d + 1)|S_U|]L_{G_0}$. Compared to [16], UR-CP-ABE achieves significantly lower key storage overhead. The latter constructs an attribute binary tree per attribute group and stores separate path-related sub-items, leading to rapid cost growth with more user attributes. UR-CP-ABE thus saves approximately $[|S_U|(d + 1)]L_{\mathbb{Z}_p} + [|S_U| - (d + 1)]L_{G_0}$.

In the ciphertext generation stage, let the minimum coverage set of the revocation list include $|N_C|$ nodes. Each ciphertext stores sub-items related to these nodes, resulting in a final storage overhead of $(1 + 2|S_T| + |N_C|)L_{G_0} + L_{G_T}$. Compared to [15], UR-CP-ABE avoids additional overhead in re-encryption: the latter adds one ciphertext sub-item per

revocation-related attribute group (to the original ciphertext) and one per user in such groups. With average attributes $|S_R|$ for revoked users and average groups $|N_S|$ per attribute, [15] incurs extra overhead of $|S_R|L_{\mathbb{Z}_p} + (|S_R||N_S|)L_{G_0}$. UR-CP-ABE has slightly more G_0 sub-items than [25]. For [26], revocation requires storing $m \cdot e(g, g)^{\alpha_s}$, plus rk_5 and C' . Storage consumption is shown in Table III.

2) *Computational Overhead*: During the experiment, the time-consuming operations are the exponential operation E and bilinear pairing operation E_p . Note that exponential operations in G_0 and G_T differ slightly in time cost, denoted as E_{G_0} and E_{G_T} respectively; these two indicators are used to relatively measure the scheme's computing performance. [15] and [16] also use symmetric encryption/decryption, with their overhead denoted as E_k and D_k respectively. The hash map's calculation overhead is approximated by two G_0 modular exponent operations ($2E_{G_0}$). Below is the calculation overhead analysis of UR-CP-ABE and comparison schemes based on the above definitions.

In the key generation stage, [15] and [16] require two key sub-items per user attribute, involving $3G_0$ modular exponentiations, while UR-CP-ABE needs only one sub-item ($1G_0$). [25] and [26] use two G_0 exponentiations per attribute sub-item. Thus, as user attributes increase, their total computation costs far exceed UR-CP-ABE's. Compared to [13], UR-CP-ABE adds two G_0 exponentiations for user path set secrets to address security issues. With the path set node count being $d + 1$ (binary tree depth d), this adds $(2d + 2)E_{G_0}$. Overall, UR-CP-ABE's computational overhead is $(8 + 2d + |S_U|)E_{G_0}$.

In the encryption stage, UR-CP-ABE's non-attribute ciphertext sub-items involve sequential G_0 modular exponentiations for minimum covering set secrets, costing $(1 + |N_C|)E_{G_0} + E_{G_T}$; attribute sub-items require $3G_0$ exponentiations per access structure attribute, totaling $3|S_T|E_{G_0}$. Its total cost is thus $(1 + 3|S_T| + |N_C|)E_{G_0} + E_{G_T}$. [13] has a similar ciphertext structure: non-attribute sub-items cost $(2|N_C|)E_{G_0} + E_{G_T}$, attribute sub-items cost $2|S_T|E_{G_0}$, with total cost $(2 + 2|S_T| + |N_C|)E_{G_0} + E_{G_T}$. For [15], non-attribute sub-items cost $E_{G_0} + E_{G_T}$; attribute sub-items require one hash map and two G_0 exponentiations per attribute ($4|S_T|E_{G_0}$), totaling $(1 + 4|S_T|)E_{G_0} + E_{G_T}$. [16] incurs extra costs for non-attribute sub-items: one pairing, one symmetric encryption, and $E_{G_0} + E_{G_T} + E_k + E_p$; ciphertext index costs $6E_{G_0}$; attribute sub-items (one hash map + two G_0 exponentiations per attribute) cost $4|S_T|E_{G_0}$. Total cost: $(7 + 4|S_T|)E_{G_0} + E_{G_T} + E_k + E_p$. [26] requires re-encryption based on rk atop basic encryption, leading to significantly higher overhead.

In decryption, UR-CP-ABE's attribute ciphertext sub-item decryption, similar to the basic scheme [27], costs $2|S_T|E_p + |S_T|E_{G_T}$. Non-attribute sub-item decryption involves intersecting nodes of the covering set, costing $3E_p$. Total cost: $(3 + 2|S_T|)E_p + |S_T|E_{G_T}$. [13] has attribute decryption costs of $2|S_T|E_p + |S_T|E_{G_T} + |S_T|E_{G_0}$ (similar to the basic structure) and non-attribute decryption costs $3E_p + 2E_{G_0}$ (two more G_0 exponentiations than UR-CP-ABE). Total: $(3 + 2|S_T|)E_p + |S_T|E_{G_T} + (2 + |S_T|)E_{G_0}$. For [15], let $|N_S|$ and $|S_R|$ denote the average attributes per revoked user and

TABLE III
STORAGE COMPARISON

Schemes	CT	SK	PK	MSK
[13]	$(2 + S_T + N_C)L_{G_0} + L_{G_T}$	$(2 + d)L_{Z_p} + (5 + S_U)L_{G_0}$	$(2 + A + 2 U)L_{G_0} + L_{G_T}$	$(2 + 2 U)L_{Z_p}$
[15]	$ S_R L_{Z_p} + (1 + 2 S_T + S_R N_S)L_{G_0} + L_{G_T}$	$(2 + 2 S_U)L_{G_0}$	$2L_{G_0} + L_{G_T}$	$L_{Z_p} + L_{G_0}$
[16]	$(5 + 2 S_T)L_{G_0} + 2L_{G_T}$	$ S_U (d + 1)L_{Z_p} + (3 + 2 S_U)L_{G_0}$	$4L_{G_0} + L_{G_T}$	$3L_{Z_p} + L_{G_0}$
[25]	$(3 + 2 S_T)L_{G_0} + L_{G_T}$	$(3 + S_U)L_{G_0}$	$(2 + A)L_{G_0} + L_{G_T}$	$(2 + A)L_{Z_p}$
[26]	$(1 + 2 S_T)L_{G_0} + 3L_{G_T}$	$(2 + S_U)L_{G_0}$	$(3 + A)L_{G_0} + L_{G_T}$	L_{G_0}
UR-CP-ABE	$(1 + 2 S_T + N_C)L_{G_0} + L_{G_T}$	$(4 + d + S_U)L_{G_0}$	$(1 + A + 2 U)L_{G_0} + L_{G_T}$	$(1 + 2 U)L_{Z_p}$

TABLE IV
COMPUTATION COMPARISON

Schemes	KeyGen	Encrypt	Decrypt	CTUpdate
[13]	$(7 + S_U)E_{G_0} + E_k$	$(2 + 2 S_T + N_C)E_{G_0} + E_{G_T}$	$(3 + 2 S_T)E_p + S_T E_{G_T} + (2 + S_T)E_{G_0}$	$ e E_{G_0}$
[15]	$(4 + 5 S_U)E_{G_0}$	$(1 + 4 S_T)E_{G_0} + E_{G_T}$	$(1 + 2 S_T + S_R)E_p + T E_{G_T} + S_R (4 + N_S)E_{G_0}$	$ S_R E_p + S_R (4 + N_S)E_{G_0}$
[16]	$(4 + 5 S_U)E_{G_0}$	$(7 + 4 S_T)E_{G_0} + E_{G_T} + E_k + E_p$	$(1 + 2 S_T)E_p + T E_{G_T} + S_R E_{G_0} + D_k$	$ S_R E_{G_0}$
[25]	$(10 + S_U)E_{G_0}$	$(4 + S_T)E_{G_0} + E_{G_T}$	$(3 + S_T)E_p + S_T E_{G_T}$	$5 S_R E_{G_0}$
[26]	$(2 + S_U)E_{G_0}$	$(2 + 3 S_T)E_{G_0} + E_{G_T}$	$(1 + 2 S_T)E_p + S_T E_{G_T}$	$(1 + 3 S_T)E_{G_0} + E_{G_T}$
UR-CP-ABE	$(8 + 2d + S_U)E_{G_0}$	$(1 + 3 S_T + N_C)E_{G_0} + E_{G_T}$	$(3 + 2 S_T)E_p + S_T E_{G_T}$	$ e E_{G_0}$

average users per attribute group, respectively. Header decryption requires: $|S_R|(E_p + 3E_{G_0})$ (decrypting attribute group secrets), $|S_R||N_S|E_{G_0}$ (restoring secrets), and $|S_R|E_{G_0}$ (attribute key sub-item exponentiation). Leaf/internal node decryption costs $2|S_T|E_p + |T|E_{G_0}$, plus one final pairing. Total: $(1 + 2|S_T| + |S_R|)E_p + |T|E_{G_T} + |S_R|(4 + |N_S|)E_{G_0}$. [16] first computes path set-covering set intersections for revoked users' attributes, restoring secrets with $|S_R|E_{G_0}$ (one G_0 exponentiation per attribute). Attribute decryption costs $2|S_T|E_p + |T|E_{G_T}$; final recovery needs one symmetric decryption and one pairing. Total: $(1 + 2|S_T|)E_p + |T|E_{G_T} + |S_R|E_{G_0} + D_k$. [26] has attribute ciphertext decryption costs similar to UR-CP-ABE.

To measure UR-CP-ABE's revocation efficiency in ciphertext re-encryption, let $|e|$ (a constant related to revocation lists and user count) denote the node jump count between the old revocation list RL and updated RL' . UR-CP-ABE and [13] only need $|e|G_0$ modular exponentiations to update minimum covering set ciphertext sub-items, with total cost $|e|E_{G_0}$. [15] first performs one G_0 exponentiation per revocation-related attribute ciphertext sub-item ($|S_R|E_{G_0}$), then evaluates ciphertext headers for each associated attribute group $|S_R|E_p + |S_R|(3 + |N_S|)E_{G_0}$. Total cost: $|S_R|E_p + |S_R|(4 + |N_S|)E_{G_0}$. [16] requires $1G_0$ exponentiation per revocation-related attribute ciphertext sub-item, totaling $|S_R|E_{G_0}$. [25] incurs an additional $5|S_R|E_{G_0}$ in re-encryption. [26] has significantly higher re-encryption overhead (including PP and $KeyGen$ updates). Performance comparison in Table IV.

B. Experiment Analysis

This experiment uses a toolkit based on [27], the JPBC library, and a 160-bit elliptic curve group constructed from a 512-bit super-singular curve with the equation $y^2 = x^3 + x$. The experimental environment is configured as follows: Windows 10 operating system platform, Core™i5-4210M (2.6GHz), 12GB of memory. To measure the impact of increasing numbers of attributes, we used the control variable method and conducted 50 rounds of experiments. The results were obtained by averaging the measurements.

1) *Computational Overhead*: In the simulation, the attribute space size is set to $|A| = 100$, and the number of user binary tree nodes is $|U| = 128$ (corresponding to a binary tree depth of $d = \lceil \log_2 128 \rceil = 7$, with a total of $2 \times 128 - 1 = 255$ nodes). User-owned attributes satisfy the access structure with an "AND" attribute relationship. For key generation, the computational cost of UR-CP-ABE is comparable to those of the schemes in [13], [25], [26], while its maximum computational cost is only approximately 21% of that in [15], [16]. Regarding ciphertext generation: the scheme in [13] is slightly more efficient; when the number of attributes in the access structure reaches the maximum, the computational overhead of UR-CP-ABE is about 71% of that in [15] and 56% of that in [16]. For decryption, when the number of user attributes is maximized, the overhead of UR-CP-ABE is 53% of that in [13] and 43% of that in [15]. In the re-encryption phase (for revoking users with 1–100 attributes), UR-CP-ABE outperforms the schemes in [15], [16], [25], [26], with a constant computational cost of approximately 8.21 ms, independent of the number of attributes owned by the revoked user. Detailed performance data are presented in Fig. 6(a)–(d).

2) *Large-Scale User*: To confirm that UR-CP-ABE is adaptable to large-scale user environments, tests center on the correlation between re-encryption time overhead and user scale. The experiment records the variation law of re-encryption overhead via test cases where the revocation list expands gradually from empty to 100 users, with results for tree depths $d = 7$ and $d = 17$ shown in Fig. 7. Meanwhile, the experiment deduces overhead performance in ultra-large-scale scenarios based on the mathematical characteristics of the binary tree structure.

In the 2^7 -user scenario, re-encryption overhead is determined solely by the number of differential nodes $|e|$ in the binary tree's minimum covering set, with no linear correlation to the number of revoked users. Phased growth only occurs when the order of $|e|$ jumps. In the binary tree user structure, tree depth d and total user count $|U|$ satisfy $d = \lceil \log_2 |U| \rceil$, and the number of differential nodes $|e|$ increases slightly at a constant level with d . When the user scale expands to $2^{17} = 131,072$ users, d only increases from $d = 7$ to $d = 17$, and $|e|$ only increases from 3~5 to 5~6. In conclusion, UR-CP-ABE's re-encryption overhead grows logarithmically with user scale. Even in ultra-large-scale

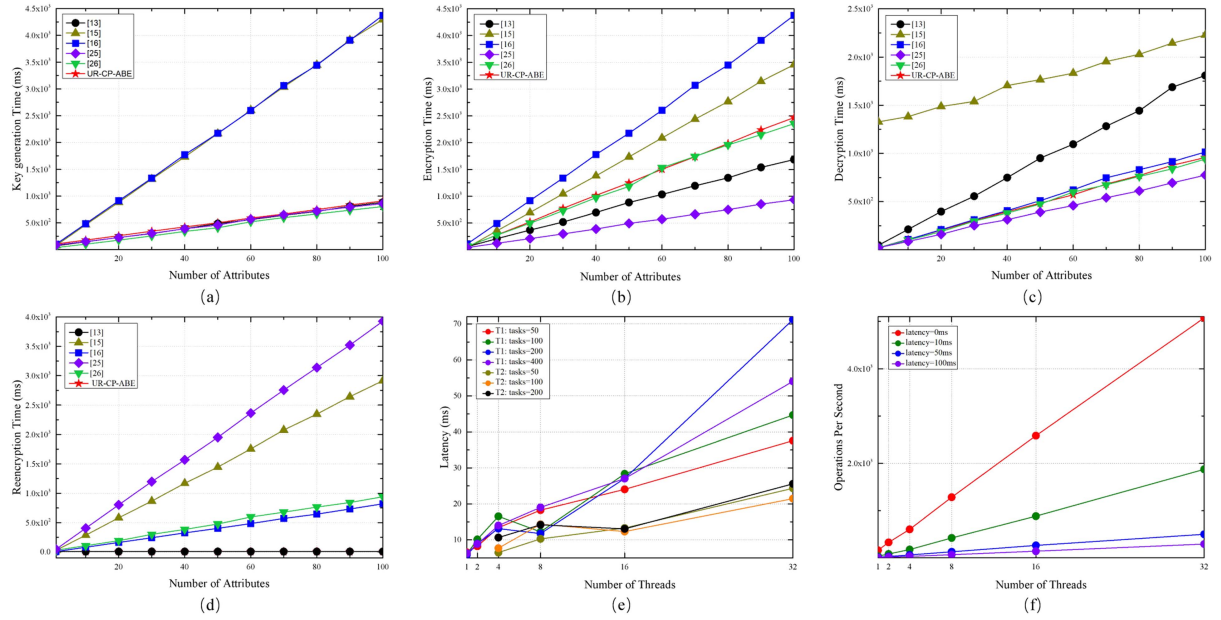


Fig. 6. Computational overhead and performance comparisons: (a) Key generation time comparison; (b) Encryption time comparison; (c) Decryption time comparison; (d) Re-encryption time comparison; (e) Concurrent revocation latency under different thread-task combinations (T1 & T2); (f) Throughput vs. network latency/thread count.

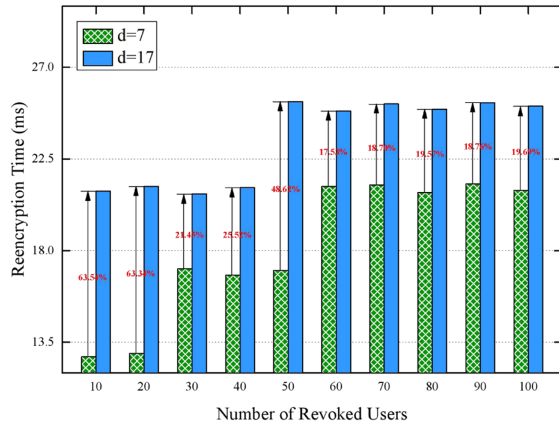


Fig. 7. Re-encryption Overhead by Revoked Users.

user environments, it maintains low overhead, fully meeting the scalability requirements of practical distributed systems for large-scale user scenarios.

3) *Concurrent Revocation Latency*: To verify UR-CP-ABE's engineering performance in dynamic concurrent environments, a concurrent revocation latency experiment was conducted with two binary tree structures: $T1$ ($d = 3$) and $T2$ ($d = 7$). The test used a “thread count-task count” dual-variable design: $T1$ had 24 combinations (threads: 1,2,4,8,16,32; tasks: 50,100,200,400) and $T2$ had 12 combinations (threads: 4,8,16,32; tasks: 50,100,200), focusing on high concurrency. Each combination was repeated 5 times to eliminate random errors. The core indicator is the total time from a thread initiating a revocation request to the *DMS* completing *RL* validity/timeliness verification, *CTUpdate*, and returning a success response.

For $T1$, latency increases moderately with thread count under fixed task volumes: 50 tasks (1→32 threads): 4.21 ms→5.28 ms (↑25.4%); 400 tasks (1→32 threads): 8.76 ms→11.39 ms (↑30.0%), indicating effective resource control without significant surges. For $T2$, latency grows linearly with task volume under fixed thread counts: 32 threads (50→200 tasks): 21.26 ms→38.71 ms (↑82.1%). Under the same “thread-task” combination, $T2$'s average latency is only 1.8–2.2x that of $T1$ —far below the 16-fold user scale increase. Maximum latency remains acceptable ($T1$: 11.39 ms; $T2$: 38.71 ms), satisfying the “low-latency revocation” requirement. Detailed performance data is provided in Fig. 6(e).

4) *Key Vault Integration*: To verify UR-CP-ABE's integration compatibility with Azure Key Vault and quantify network latency/concurrency impacts on system throughput, a dedicated simulation test is conducted. Specified network latency is simulated before each key call request, plus 5 ms default server processing time to replicate real request flows, with core metrics monitoring throughput (ops/sec) and request latency. The experiment adopts a “network latency-thread count” dual-variable control design: thread counts are 1, 2, 4, 8, 16, 32 (covering low-to-high concurrency), simulated network latency values are 0 ms, 10 ms, 50 ms, 100 ms (covering typical cloud conditions), and each thread executes 100 fixed tasks.

In the 0 ms baseline scenario, throughput increases approximately linearly with thread count, reaching 5076.64 ops/sec at 32 threads—31.0x that of 1 thread (163.65 ops/sec)—confirming no significant integration-induced local overhead. For low-to-medium latency (10 ms–50 ms), 16-thread throughput hits 885.20 ops/sec and 266.01 ops/sec (22.7% and 69.7% below baseline), compatible with cross-regional cloud latency. At 100 ms high latency, 32-thread throughput drops to 295.00 ops/sec (5.8% of baseline) with no task failures. Analysis shows

a strong negative correlation between throughput and network latency ($R^2 = 0.97$); with latency ≤ 50 ms, throughput remains above 260 ops/sec, meeting high concurrency requirements. In summary, UR-CP-ABE exhibits excellent compatibility with Azure Key Vault, with controllable latency impact on throughput, making it suitable for large-scale distributed cloud environments. Detailed performance data is provided in Fig. 6(f).

VII. CONCLUSION

To address the limitations of CP-ABE schemes in user permission revocation, we propose a binary-tree-based revocation mechanism and design UR-CP-ABE, a ciphertext-policy ABE scheme with efficient user revocation. By leveraging the binary tree data structure, UR-CP-ABE enhances user revocation efficiency, ensuring re-encryption overhead remains constant regardless of the number of revoked user attributes. To resist user-collusion attacks, the scheme blinds originally exposed secret parameters without compromising performance, thereby providing stronger security. In addition, UR-CP-ABE enables efficient and secure bidirectional revocation, supporting both the revocation and the restoration of user access rights. Theoretical analysis and experimental results confirm that UR-CP-ABE outperforms existing schemes in decryption and re-encryption efficiency.

REFERENCES

- [1] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2005, pp. 457–473.
- [2] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 89–98.
- [3] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 321–334.
- [4] A. Boldyreva, V. Goyal, and V. Kumar, "Identity-based encryption with efficient revocation," in *Proc. 15th ACM Conf. Comput. Commun. Secur.*, 2008, pp. 417–426.
- [5] M. Horváth, "Attribute-based encryption optimized for cloud computing," in *Proc. 41st Int. Conf. Curr. Trends Theory Pract. Comput. Sci.*, 2015, pp. 566–577.
- [6] N. Attrapadung and H. Imai, "Conjunctive broadcast and attribute-based encryption," in *Proc. 3rd Int. Conf.*, 2009, pp. 248–265.
- [7] Z. Xu and K. M. Martin, "Dynamic user revocation and key refreshing for attribute-based encryption in cloud storage," in *Proc. IEEE 11th Int. Conf. Trust Secur. Privacy Comput. Commun.*, 2012, pp. 844–849.
- [8] A. Balu and K. Kuppusamy, "Ciphertext-policy attribute-based encryption with user revocation support," in *Proc. Int. Conf. Heterogeneous Netw. Qual. Rel. Secur. Robustness*, 2013, pp. 696–705.
- [9] P. K. Tysowski and M. A. Hasan, "Hybrid attribute-and re-encryption-based key management for secure and scalable mobile applications in clouds," *IEEE Trans. Cloud Comput.*, vol. 1, no. 2, pp. 172–186, Jul./Dec. 2013.
- [10] Y. Shi, Q. Zheng, and J. Liu, "Directly revocable key-policy attribute-based encryption with verifiable ciphertext delegation," *Inf. Sci.*, vol. 295, pp. 221–231, 2015.
- [11] Z. Liu and D. S. Wong, "Practical attribute-based encryption: Traitor tracing, revocation and large universe," *Comput. J.*, vol. 59, no. 7, pp. 983–1004, 2016.
- [12] H. Ma, T. Peng, and Z. Liu, "Directly revocable and verifiable key-policy attribute-based encryption for large universe," *Int. J. Netw. Secur.*, vol. 19, no. 2, pp. 272–284, 2017.
- [13] Z. Liu, S. Duan, and P. Zhou, "Traceable-then-revocable ciphertext-policy attribute-based encryption scheme," *Future Gener. Comput. Syst.*, vol. 93, pp. 903–913, 2019.
- [14] Y. Yasumura, H. Imabayashi, and H. Yamana, "Attribute-based proxy re-encryption method for revocation in cloud storage: Reduction of communication cost at re-encryption," in *Proc. IEEE 3rd Int. Conf. Big Data Anal.*, 2018, pp. 312–318.
- [15] G. Ramu, B. E. Reddy, and A. Jayanthi, "Fine-grained access control of EHRs in cloud using CP-ABE with user revocation," *Health Technol.*, vol. 9, no. 4, pp. 487–496, 2019.
- [16] W. Zhao, X. Dong, and Z. Cao, "A revocable publish-subscribe scheme using CP-ABE with efficient attribute and user revocation capability for cloud systems," in *Proc. IEEE 2nd Int. Conf. Electron. Commun. Eng.*, 2019, pp. 31–35.
- [17] H. Lian, Q. Wang, and G. Wang, "Large universe ciphertext-policy attribute-based encryption with attribute level user revocation in cloud storage," *Int. Arab J. Inf. Technol.*, vol. 17, no. 1, pp. 107–117, 2020.
- [18] K. Sethi, A. Pradhan, and P. Bera, "PMTER-ABE: A practical multi-authority CP-ABE with traceability, revocation and outsourcing decryption for secure access control in cloud systems," *Cluster Comput.*, vol. 24, pp. 1525–1550, 2021.
- [19] J. Hao, W. Tang, and C. Huang, "Secure data sharing with flexible user access privilege update in cloud-assisted IoMT," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 933–947, Second Quarter 2022, doi: 10.1109/tetc.2021.3052377.
- [20] S. Deng, G. Yang, and W. Dong, "Flexible revocation in ciphertext-policy attribute-based encryption with verifiable ciphertext delegation," *Multimedia Tools Appl.*, vol. 82, no. 14, pp. 22251–22274, 2023.
- [21] S. Chen, J. Li, and Y. Zhang, "Efficient revocable attribute-based encryption with verifiable data integrity," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10441–10451, Mar. 2024.
- [22] Y. Lu, T. Feng, and C. Liu, "A blockchain and CP-ABE based access control scheme with fine-grained revocation of attributes in cloud health," *CMC-Comput. Mater. Continua*, vol. 78, no. 2, pp. 2787–2811, 2024.
- [23] X. Liang, Y. Liu, and J. Ning, "An access control scheme with privacy-preserving authentication and flexible revocation for smart healthcare," *IEEE J. Biomed. Health. Inf.*, vol. 28, no. 6, pp. 3269–3278, Jun. 2024.
- [24] X. Li, G. Yang, and T. Xiang, "Make revocation cheaper: Hardware-based revocable attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 100–100.
- [25] R. Sarma and S. Moulik, "e-SAFE: A secure and efficient access control scheme with attribute convergence and user revocation in fog enhanced IoT for E-health," *J. Inf. Secur. Appl.*, vol. 85, 2024, Art. no. 103859.
- [26] C. Ge, W. Susilo, and Z. Liu, "Attribute-based proxy re-encryption with direct revocation mechanism for data sharing in clouds," *IEEE Trans. Dependable Secur. Comput.*, vol. 21, no. 2, pp. 949–960, Mar./Apr. 2024.
- [27] B. Waters, "Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization," in *Proc. Int. Workshop Public Key Cryptography*, 2011, pp. 53–70.
- [28] D. Boneh, R. Canetti, and S. Halevi, "Chosen-ciphertext security from identity-based encryption," *SIAM J. Comput.*, vol. 36, no. 5, pp. 1301–1328, 2007.



Zhen Guo received MS degree from the College of Computer Science, Sichuan Normal University, China. She is currently working toward the PhD degree with the School of Computer Science, Sichuan University, China, and concurrently serves with the Office of Informatization Construction and Management, Sichuan University. Her research interests include outlier detection, data mining, and information security.



Jiangkai Gao is currently working toward the master's degree with the School of Computer Science, Sichuan Normal University, Chengdu, China. His research focuses on secure and privacy-preserving systems, with an emphasis on policy-based access control models, cryptographic enforcement mechanisms, and formal methods for privacy assurance in networked and data-intensive environments.



Shuainan Liu received the MS degree in computer science and technology, Sichuan Normal University, in 2021. He is currently working toward the PhD degree with Texas Tech University. His research interests include differential privacy, federated learning, and cryptography.



Rong Wang received the PhD degree with the School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu, China, in 2021. She is currently a lecturer with the College of Computer Science, Sichuan Normal University, Chengdu, China. Her research interests include data privacy, data mining, and artificial intelligence security.



Chaosheng Feng received the PhD degree in information and communication engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2010. He is currently a professor and doctor tutor with the Department of Computer Science, Sichuan Normal University, Sichuan, China. He was with the Postdoctoral Mobile Station, University of Electronic Science and Technology of China and with the postdoctoral Workstation, Chengdu HighTech Zone, from 2010 to 2012.

His research interests include cloud computing, privacy protection, and data security.

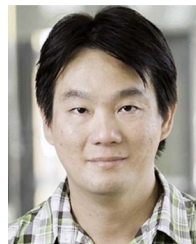


Keping Yu (Senior Member, IEEE) received the ME and PhD degrees from the Graduate School of Global Information and Telecommunication Studies, Waseda University, Shinjuku, Japan, in 2012 and 2016, respectively. He was a research associate, junior researcher, and researcher with Global Information and Telecommunication Institute, Waseda University, from 2015 to 2019, 2019 to 2020, and 2020 to 2022, respectively. He is currently an associate professor, vice director of the Institute of Integrated Science and Technology, and director of the Yielding Ubiquitous

Learning for Adaptive Broadband Laboratory, Hosei University, Tokyo, Japan. He has authored more than 200 peer-reviewed research papers and books, including more than 100 IEEE/ACM Transactions papers.

Dr. Yu has been a Highly Cited researcher identified by Clarivate™ (2023) and the World's Top 2% Scientists identified by Stanford University from 2022 to 2025. He was the recipient of the 5th Japan Research Front Award from Clarivate Analytics Japan, Best Symposium Award from IWCMC 2023, IEEE Outstanding Leadership Award from IEEE BigDataSE 2021, Best Paper Award from IEEE Consumer Electronics Magazine Award 2022 (1st Place Winner), IEEE ICFTIC 2021, ITU Kaleidoscope 2020, and Student Presentation Award from JSST 2014.

He has hosted and participated in more than 10 projects, is involved in many standardization activities organized by ITU-T and ICNRG of IRTF, and has contributed to ITU-T Standards Y.3071 and Supplement 35. He has been the guest editor of *IEEE Transactions on Computational Social Systems*, *IEEE Journal of Biomedical and Health Informatics*, and *Renewable & Sustainable Energy Reviews*. He was the general co-chair and publicity co-chair of the IEEE VTC2020-Spring 1st EBTSSRA workshop, general co-chair of IEEE ICC2020 2nd EBTSSRA workshop, general co-chair of IEEE TrustCom2021 3rd EBTSSRA Workshop, and session chair of IEEE ICC2020 and ITU Kaleidoscope 2016.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the PhD degree in information security from the Queensland University of Technology, Australia, in 2006. He is currently a Cloud Technology Endowed professor with the University of Texas at San Antonio. He was a recipient of the 2022 IEEE Hyper-Intelligence TC Award for Excellence in Hyper Intelligence Systems (Technical Achievement award), the IEEE Systems, Man, and Cybernetics TC on Homeland Security Research and Innovation Award in 2022, the 2022 IEEE TC on Secure and Dependable Measurement Mid-Career Award, and the 2019 IEEE TC on Scalable Computing Award for excellence in scalable computing (middle career researcher). He is the Founding Chair of IEEE TEMS TC on Blockchain and Distributed Ledger Technologies. He is the Founding co-editor-in-chief of *ACM Distributed Ledger Technologies: Research and Practice*.



Mohsen Guizani (Fellow, IEEE) received the BS degree (with Distinction), MS, and PhD degrees in electrical and computer engineering from Syracuse University, Syracuse, NY, USA. He is currently a professor of machine learning with the Mohamed Bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE. Previously, he worked with different institutions in USA. He is the author of 11 books, more than 1000 publications and several US patents. His research interests include applied machine learning, artificial intelligence, smart city, Internet of Things, intelligent autonomous systems, and cybersecurity.

intelligent autonomous systems, and cybersecurity.